

Spec Driven Development

Teoria del libro SDDEquiposAgiles, herramienta OpenSpec CLI y metodologia aplicada a Flutter + Clean Architecture + Supabase.
La especificacion como contrato; el cambio como unidad de trabajo.

TEORIA

**SDDEquiposAgiles_v.1 ·
Fases · Puertas**

HERRAMIENTA

**OpenSpec CLI · EARS ·
Deltas**

STACK

**Flutter · Clean Arch ·
Supabase**

NIVEL

Intermedio a Avanzado

SDD

OpenSpec

EARS

Flutter

Supabase

Clean Architecture

Contenido

32 capítulos · 32 archivos · ~6-8 horas de lectura

PARTE 1 · NUCLEO DEL MODULO

00	02 - Spec Driven Development (SDD)
01	01 - Teoría SDD: Mapa del Libro Oficial
02	02 - SDD Aplicado a Flutter + Supabase
03	03 - OpenSpec: Guía Práctica
04	04 - Plantilla de Cambio OpenSpec
05	05 - Referencia Rápida
06	06 · Auditoría de código generado por IA (Modo completo)

PARTE 2 · EJEMPLOS DE CAMBIOS

E0	Ejemplos de Cambios OpenSpec
EC1a	Proposal: add-cart
EC1b	Spec: shopping-cart
EC1c	Design: add-cart
EC1d	Tasks: add-cart
EC2a	Proposal: approve-reservas
EC2b	Spec: appointments
EC2c	Design: approve-reservas
EC2d	Tasks: approve-reservas
EC3a	Proposal: add-elearning-progress
EC3b	Spec: enrollment-progress
EC3c	Design: add-elearning-progress
EC3d	Tasks: add-elearning-progress
EC4a	Proposal: add-facturacion
EC4b	Spec: invoicing
EC4c	Design: add-facturacion
EC4d	Tasks: add-facturacion
EC5a	Proposal: add-delivery-seguimiento
EC5b	Spec: order-lifecycle + delivery-tracking
EC5c	Design: add-delivery-seguimiento
EC5d	Tasks: add-delivery-seguimiento
EC6a	Proposal: add-buyers
EC6b	Spec: buyers-management
EC6c	Design: add-buyers

02 - Spec Driven Development (SDD)

Escribe la spec antes que el código. Deja que la IA ejecute tu contrato, no tus ideas sueltas.

Qué es este módulo

La metodología **completa** de Spec Driven Development del curso, en tres piezas:

Pieza	Qué aporta
 Teoría	El libro pdf/SDDEquiposAgiles_v.1.pdf (Scrum Manager, 74 págs.) + resumen pdf/Guia-SDD-equipos-agiles.pdf
 Herramienta	OpenSpec — specs markdown vivas en el repo, ejecutables por 30+ agentes IA
 Metodología aplicada	Este módulo: el flujo SDD operativizado para Flutter + Supabase + Clean Architecture

Nota histórica: este módulo absorbe y reemplaza al antiguo `02-DISEÑO-FEATURE`. La carpeta original se conserva como referencia histórica; toda la teoría de diseño de features vive ahora aquí, en terminología SDD estándar (tabla de equivalencias).

¿Por qué SDD?

Problema	Solución
Vibe coding falla a escala: 19% más lento pese a percibir un 20% más rápido (METR, 2025)	SDD: spec antes, código después
La IA inventa decisiones que no especificaste	Spec como contrato explícito
No sabes cuándo usar SDD y cuándo no	Principio de proporcionalidad
El código heredado se rompe con cambios mal planificados	Impact Report brownfield
Las specs se desactualizan y nadie las consulta	Clarity Gate + specs vivas
No tienes una herramienta concreta para implementar SDD	OpenSpec (CLI ligera, sin API keys)

Índice

#	Archivo	Descripción
01	01-teoria-sdd.md	Mapa de lectura del libro oficial cap por cap → aplicación a tu stack
02	02-sdd-flutter-supabase.md	Guía aplicada: Impact Report, 4 fases, 3 puertas, EARS, oleadas, boundaries
03	03-openspec-guia-practica.md	Setup y workflow de OpenSpec en un proyecto Flutter
04	04-plantilla-cambio-openspec.md	Plantilla lista para copiar: proposal, spec delta, design, tasks
05	05-referencia-rapida.md	Cheat sheet + glosario de equivalencias → SDD
06	06-auditoria-codigo-ia.md	Checklist para auditar código escrito por IA (Modo completo): contratos, sealed states, RLS, tests contra EARS

#	Archivo	Descripción
07	07-guia-paso-a-paso.md	Receta de cocina: cada fase de SDD con su comando OpenSpec, qué escribir, y checklist de puertas
—	<code>ejemplos-cambios/</code>	6 cambios completos listos para copiar (carrito walkthrough incluido)
—	<code>trabajar-sin-ia/</code>	Ejercicios de diseño con papel y lápiz, sin herramientas IA
—	<code>pdf/</code>	Libro oficial (SDDEquiposAgiles_v.1) + guías generadas del módulo (<code>02-SPEC-DRIVEN-DEVELOPMENT.pdf</code> y <code>GUIA-RESUMEN-*.pdf</code> , regenerables con <code>pdf/src/build_pdf.py</code>)

Ruta de aprendizaje

1. Lee el libro (Partes II y III primero)	→ <code>01-teoria-sdd.md</code> es tu mapa
2. Aprende la herramienta	→ <code>03-openspec-guia-practica.md</code>
3. Domina la metodología en tu stack	→ <code>02-sdd-flutter-supabase.md</code>
4. Aplica paso a paso con OpenSpec	→ <code>07-guia-paso-a-paso.md</code>
5. Practica sin IA	→ <code>trabajar-sin-ia/</code>
6. Copia un ejemplo real y adapítalo	→ <code>ejemplos-cambios/add-cart/</code>
7. Usa la plantilla en tu proyecto	→ <code>04-plantilla-cambio-openspec.md</code>
8. Delega todo y verifica como auditor	→ <code>06-auditoria-codigo-ia.md</code>

Módulos relacionados

Módulo	Conexión
01 - Clean Architecture	La estructura que las specs especifican capa por capa
02 - Diseño de Feature (histórico)	Origen de este módulo; conservado como archivo histórico
10 - Makefile	Targets para automatizar verificación de specs
12 - Git Flow y Conventional Commits	Commits atómicos por tarea; specs conviven con convenciones
22 - Diseño de Sistemas	Escala de feature (SDD) a sistema completo

Nivel: Intermedio-Avanzado **Tiempo estimado:** 4-6 horas (incluyendo lectura parcial del libro) **Requisito previo:** Conocer la estructura Clean Architecture (`01-CLEAN-ARCHITECTURE`)

01 - Teoría SDD: Mapa del Libro Oficial

La teoría de este módulo es el libro [SDDEquiposAgiles_v.1.pdf](#) (Scrum Manager, v1.0 — abril 2026, 74 páginas). Este archivo NO lo duplica: te da el **mapa de lectura** y traduce cada capítulo a tu stack (Flutter + Supabase + Clean Architecture).

Cómo usar este archivo

- 1. Lee el libro una vez completa ([pdf/SDDEquiposAgiles_v.1.pdf](#))
 - 2. Usa este mapa como índice de consulta rápida: ¿en qué capítulo estaba X?
 - 3. Cada fila te dice dónde se **operativiza**: qué archivo de este módulo convierte ese concepto en algo ejecutable
- El complemento [pdf/Guia-SDD-equipos-agiles.pdf](#) (35 págs.) es el resumen *state-of-the-art* del mismo contenido: útil como repaso exprés, no como lectura principal.

Mapa maestro del libro → este módulo

Cap	Tema del libro	Idea clave	Se operativiza en
1-3	Del vibe coding a SDD · spec como artefacto primario · problemas estructurales	La spec precede al código; el agente es un ejecutor con contrato, no un interlocutor	README (filosofía)
4-5	SDD y agilidad · flujo de cuatro fases	Requisitos → Diseño → Tareas → Implementación; revisar en puertas, no durante	02-sdd-flutter-supabase.md §flujo
7	Fase 1: Requisitos	Impact Report ANTES de requisitos · historias precisas · notación EARS	02 §Paso 0 y Fase 1
8	Fase 2: Diseño	Ficheros afectados · decisiones explícitas · heurística sénior	02 §Fase 2
9	Fase 3: Tareas	Tareas atómicas (rol/tarea/restricciones/criterios) · oleadas · descomposición justa	02 §Fase 3
10	Fase 4: Implementación	Ejecución delegada · commits atómicos · revisión al cierre de fase	02 §Fase 4
11	Puertas de aprobación	3 puertas + revisión final; el coste del error crece por fase; quién aprueba	02 §Puertas
12	Anatomía de una buena spec	5 principios de Osmani · 6 áreas del análisis de GitHub	04-plantilla
13	Boundaries Always / Ask First / Never	Marco de delegación en 3 niveles, evolutivo	02 §Boundaries
14	Maldición de las instrucciones	Más instrucciones = menor cumplimiento individual; dividir, no acumular	05-referencia-rapida.md
15	Specs vivas vs estáticas	spec-first / spec-anchored / spec-as-source; actualizar en el mismo commit	02 §Specs vivas
16-18	SDD en el equipo ágil	Equivalencias Scrum ↔ SDD; roles; ceremonias	05-referencia-rapida.md §equivalencias
19	Brownfield	Impact Report obligatorio · formato delta · adopción gradual · constitución del proyecto	02 §Paso 0
20-21	Herramientas 2026 · nativo vs framework	La vía intermedia: skills y slash commands	03-openspec-guia-practica.md

Cap	Tema del libro	Idea clave	Se operativiza en
22	Métricas	Métricas de flujo y de resultado; qué NO medir	05-referencia-rapida.md \$métricas
23	Anti-patrones	Sobreespecificación · documentación zombi · teatro de especificación · SDD como control	05-referencia-rapida.md \$antipatrones

Los 9 conceptos nucleares (resumen de 1 línea)

Concepto	En una frase
Las 4 fases	Requisitos (qué) → Diseño (cómo) → Tareas (en qué orden) → Implementación (ejecutar).
Las 3 puertas	Puntos donde el humano aprueba entre fases; corregir ahí es barato, después es caro.
EARS	5 patrones (ubicuo, evento, estado, no deseado, opcional) que eliminan ambigüedad sin perder legibilidad.
Tareas atómicas	1–3 ficheros, prompt con rol/tarea/restricciones/criterios, verificable en aislado.
Oleadas	Tareas independientes en paralelo (subagentes con contexto limpio); oleadas en secuencia.
Commits atómicos	1 tarea = 1 commit: reversibilidad granular, trazabilidad y bisección eficaz.
Boundaries	Always (sin preguntar) / Ask First (aprobar antes) / Never (líneas rojas).
Formato delta	La spec describe solo lo que cambia: ADDED / MODIFIED / REMOVED (popularizado por OpenSpec).
Clarity Gate	¿Un agente distinto regeneraría código equivalente solo con la spec? Si no, faltan supuestos.

Lo que el libro NO trae (y vive en este módulo)

El libro es genérico (cualquier lenguaje/plataforma). Lo específico de tu stack que este módulo aporta:

Específico de Flutter + Supabase	Dónde
Impact Report sobre un codebase Clean Architecture (features, contratos, DI, rutas)	02 \$Paso 0
Clasificación de requisitos EARS por componente Clean Arch (entity, repository, usecase, datasource, cubit, page)	02 \$Fase 1
Reglas de seguridad como políticas RLS en Supabase (escenarios RS)	02 \$Fase 2
Oleadas estándar de implementación para una feature (entity → contratos → model/datasource → impl/cubit → pages/DI/router → tests)	02 \$Fase 3
Boundaries concretos (pubspec, migraciones, RLS de producción, build.gradle...)	02 \$Boundaries
Ejecución con la skill <code>clean-arch-feature</code> o con agente + OpenSpec	02 \$Fase 4 + 03

Constitución del proyecto (prerrequisito brownfield)

Antes de tu primer cambio SDD en un proyecto existente, asegúrate de tener la **constitución** del proyecto (cap 19 del libro): un `AGENTS.md` / `CLAUDE.md` con stack, estructura de carpetas, patrones establecidos y flujo git. OpenSpec genera el stub automáticamente con `openspec init` (ver guía práctica). Sin ella, cada spec se escribe en el vacío.

Referencias

- 📖 Libro oficial: `pdf/SDDEquiposAgiles_v.1.pdf` — lee primero las Partes II y III (caps 6–15)
- 📄 Resumen state-of-the-art: `pdf/Guia-SDD-equipos-agiles.pdf`
- 🔧 Herramienta: `03-openspec-guia-practica.md`

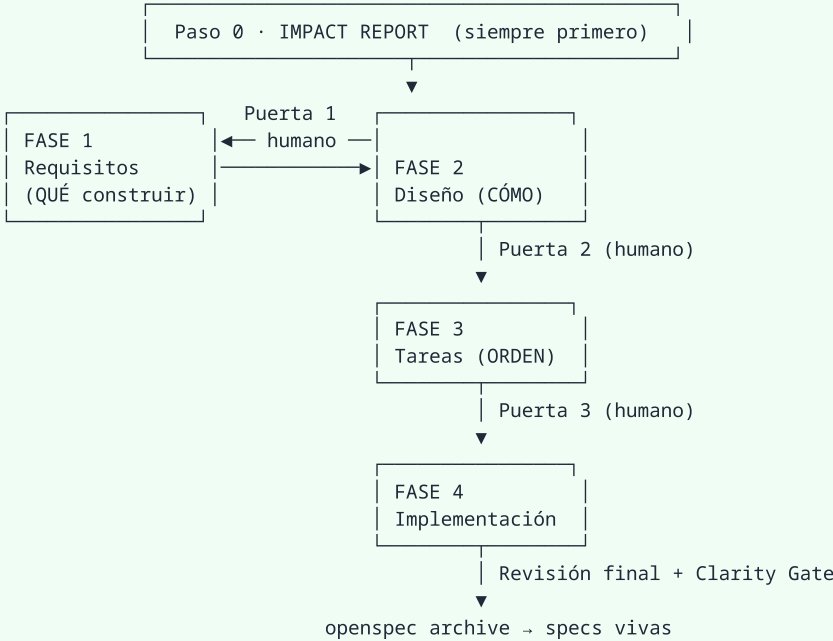
-  Metodología aplicada: [02-sdd-flutter-supabase.md](#)

02 - SDD Aplicado a Flutter + Supabase

La metodología del libro (01-teoria-sdd.md) operativizada para tu stack: Flutter + Clean Architecture + BLoC/Cubit + fpdart + GetIt + GoRouter + Supabase.

Este documento absorbe la metodología de diseño del módulo histórico 02-DISEÑO-FEATURE y la reescribe en terminología SDD estándar.

El flujo completo



Regla de oro: el agente no cruza una puerta sin aprobación humana. Corregir un requisito en la Puerta 1 cuesta minutos; corregirlo tras la implementación cuesta días.

Herramienta: cada fase vive en archivos concretos dentro de un *cambio* OpenSpec:

Fase	Archivo	Contenido
0 + 1	proposal.md	Impact Report resumido, problema, solución, alcance, actores
1 (detallado)	specs/{capacidad}/spec.md	Requisitos EARS + escenarios, formato delta
2	design.md	Ficheros afectados, contratos Dart, flujo, backend Supabase, decisiones
3	tasks.md	Tareas atómicas agrupadas en oleadas
4	código + commits atómicos	Ejecución tarea por tarea

Plantilla completa: 04-plantilla-cambio-openspec.md

Paso 0 · Impact Report (obligatorio en brownfield)

Todo proyecto real es brownfield: ya hay código. Antes de escribir UN solo requisito, analiza qué existe. (El libro lo exige en su cap. 19; aquí está adaptado a un proyecto Clean Architecture.)

Tres preguntas:

1. ¿Qué features existentes se ven afectadas?
2. ¿Qué puede reutilizarse (contratos, widgets, servicios, tablas)?
3. ¿Qué riesgos introduce el cambio?

Checklist específico de tu codebase:

- [] Features similares: lib/features/*/ – ¿existe algo parecido? ¿patrón a copiar?
- [] Entidades/contratos reutilizables: lib/features/*/domain/
- [] Tablas Supabase afectadas: esquema, políticas RLS existentes
- [] RPCs / vistas que consumen o alimentan
- [] Registro DI: lib/core/di/service_locator.dart – ¿nuevos registros?
- [] Rutas: lib/core/router/app_router.dart – ¿nueva pantalla? ¿guards?
- [] Widgets/servicios compartidos: lib/core/ y lib/shared/
- [] Convenciones: pubspec.yaml (paquetes disponibles), lints, naming
- [] Migraciones previas: supabase/migrations/ – numeración siguiente

Salida: sección "Impacto" al inicio de `proposal.md` (5–15 líneas). Si el report revela que la feature toca 3+ dominios → proporcionalidad Compleja (ver [§Proporcionalidad](#)).

Dos modos de crear el cambio

Toda feature nace como carpeta de cambio. Hay dos formas de llenarla, y **no son excluyentes** — puedes empezar artesano y terminar copiloto (o al revés):

	Modo artesano (tú propones)	Modo copiloto (la IA propone)
Comando de arranque	<code>/opsx-new-change</code> o copiar <code>04-plantilla-cambio-openspec.md</code>	<code>/opsx-propose <nombre></code>
Primera versión de la spec	La escribes TÚ, sin código	La redacta la IA desde tu descripción
Rol de la IA	Refina: busca ambigüedades, requisitos no verificables, supuestos ocultos	Propone desde cero y tú apruebas
Cómo refina	Conversación + <code>/opsx-update-change</code>	Conversación sobre su propia propuesta
Validación común	<code>openspec validate <cambio> --strict</code> + Clarity Gate	idem
Ideal para	Aprender, dominio crítico, diseño difícil, entrenar criterio	Velocidad, CRUD conocido, brownfield

Flujo del modo artesano:

- `/opsx-new-change` (o copia la plantilla) → esqueleto `proposal.md` + `specs/` + `design.md` + `tasks.md` listo
- Llenas tú: proposal, requisitos EARS y design. Piensas primero — igual que se hacía con la hoja de diseño del método anterior
- Pides revisión a la IA: *"revisame la spec: ¿qué escenario es ambiguo, qué requisito no es testeable, qué supuesto falta?"* → aplicas las mejoras con `/opsx-update-change`
- `openspec validate <cambio> --strict`
- Clarity Gate como prueba final: ¿otro agente, leyendo SOLO tu spec, produciría código equivalente?

Ambos modos convergen en la misma **Puerta 1**. La implementación posterior usa siempre el mismo motor — `/opsx-apply-change` —; lo único que varía es el **Modo** declarado en `tasks.md` (`andamiaje` | `completo`), independiente del modo de redacción elegido.

Fase 1 · Requisitos — QUÉ construir

Diluye los pasos **Alcance**, **Formular**, **Actorizar**, **Descomponer**, **Entidades** y **Reglas** del método anterior. Nada se pierde: cada pieza tiene su lugar exacto.

1.1 Historias precisas (antes "Formular")

Una historia precisa define: **actor** + **acción** + **objeto** + **propósito** + contexto.

- × Vaga: "El usuario podrá gestionar su carrito"
- ✓ Precisa: "Como cliente con sesión iniciada, quiero agregar productos a mi carrito para acumular mi pedido antes de pagar."

Prueba de precisión: ¿puedo escribir un test de aceptación sin preguntar nada más?

1.2 Actores y permisos (antes "Actorizar")

Tabla de actores con lo que pueden y NO pueden hacer. En Supabase esto mapea 1:1 a políticas RLS.

Actor	Tipo	Puede	No puede
Cliente	Primario (auth.uid())	CRUD sus items, aplicar cupón	Ver carritos ajenos
Admin	Secundario (claim role)	Ver carritos abandonados	Modificar carrito ajeno
Inventario	Sistema interno	Validar stock vía RPC	—

1.3 Alcance (antes "Paso Alcance")

Va directo a `proposal.md`: incluye / no incluye / dependencias / suposiciones / preguntas abiertas. Las preguntas abiertas se cierran ANTES de la Puerta 1 o se documentan como decisión pendiente.

1.4 Operaciones atómicas → requisitos dirigidos por evento (antes "Descomponer")

Enumera operaciones de una sola cosa, agrúpalas por actor, ordena dependencias. Luego **cada operación se convierte en un requisito** escrito en EARS.

1.5 Notación EARS (reemplaza al formato libre RN/RT/RS)

EARS elimina ambigüedad con 5 patrones. Cada requisito tiene ID único (`REQ-XXX`), nombre corto, cuerpo EARS y escenarios.

Patrón	Plantilla	Uso típico en tu stack
Ubicuo	"El sistema DEBERÁ <acción>"	Reglas invariables (ej: precio unitario congelado al agregar)
Dirigido por evento	"CUANDO <disparador> EL sistema DEBERÁ <acción>"	Interacciones UI, eventos Cubit
Dirigido por estado	"MIENTRAS <estado> EL sistema DEBERÁ <acción>"	Estados del carrito (vacío, con items)
No deseado	"SI <condición> ENTONCES el sistema DEBERÁ <respuesta>"	Errores, validaciones, failures
Opcional	"DONDE <característica> EL sistema DEBERÁ <acción>"	Feature flags

Ejemplo aplicado:

```
### Requirement: Agregar producto al carrito
El cliente agregará productos validando stock y límites.

#### Scenario: Agregar producto con stock disponible
- **WHEN** el cliente agrega un producto con stock > 0
- **THEN** el sistema DEBERÁ agregarlo con cantidad 1 y congelar su precio unitario
- **AND** recalcular subtotal, impuesto y total

#### Scenario: Producto sin stock
- **IF** el stock del producto es 0
- **THEN** el sistema DEBERÁ mostrar "Producto {nombre} sin stock disponible" y no modificar el carrito

#### Scenario: Límite de items alcanzado
- **IF** el carrito ya contiene 50 productos distintos
- **THEN** el sistema DEBERÁ mostrar "Has alcanzado el límite de 50 productos"
```

Nota: Los escenarios EARS son tus criterios de aceptación.

Cada `#### Scenario:` en `spec.md` define una condición verificable que el sistema DEBERÁ cumplir. En terminología clásica de QA/BA, estos son los *acceptance criteria* de la user story. La diferencia es el formato: EARS reemplaza la prosa

libre por patrones con keywords (WHEN/THEN , IF/THEN , GIVEN/WHEN/THEN) que eliminan ambigüedad.

Ubicación: los criterios de aceptación viven en spec.md , dentro de cada requisito (REQ-XXX), como escenarios con caso feliz + error + bordes. Los tests (unit, widget, integration) validan estos mismos criterios.

1.6 Clasificación de reglas → dónde viven

Las categorías RN/RT/RS anteriores se diluyen así:

Categoría anterior	Ahora es...	Dónde se especifica
RN (negocio): restricción/cálculo/validación/flujo	Requisitos EARS ubicuos/evento/no deseado	specs/carrito/spec.md
RT (técnicas)	Decisiones explícitas + requisitos de comportamiento interno	design.md §Decisiones
RS (seguridad)	Escenarios EARS + política RLS concreta	spec.md escenario + design.md §Backend

Reglas de negocio: no son un documento separado.

Las reglas de negocio (antes "RN") son requisitos EARS que especifican invariantes, cálculos y validaciones. Ejemplos de mapeo:

Regla de negocio	Patrón EARS	Ubicación
"El precio unitario se congela al agregar al carrito"	Ubicuo (DEBERÁ)	spec.md REQ + entity (invariante)
"Tope de descuento: 50%"	No deseado (IF/THEN)	spec.md REQ + usecase (validación)
"Stock se descuenta al confirmar pago, no al iniciar"	Ubicuo (DEBERÁ)	spec.md REQ + usecase (orquestración)
"RLS: cada usuario solo ve sus datos"	Aislamiento (GIVEN/WHEN/THEN)	spec.md escenario + design.md §Backend

Dónde vive la implementación: domain/entity (invariantes, cálculos puros) y domain/usecases (validaciones de negocio). **NUNCA** en presentation (cubit/widget).

1.7 Specs por componente Clean Architecture

Un mismo requisito de negocio genera requisitos derivados por capa. Especifica el comportamiento esperado de cada componente (no su implementación):

Componente	Qué se especifica
Entity (domain)	Invariantes, cálculos puros (subtotal, impuesto), igualdad
Repository interface (domain)	Firma de métodos, tipo de retorno Either<Failure, T>
Model (data)	Serialización JSON ↔ entity, campos nulos, snake_case ↔ camelCase
DataSource (data)	Llamadas exactas a Supabase (tabla/RPC/auth), manejo de errores de red
Repository impl (data)	Mapeo Failure ← excepciones, aplicación de RT
UseCase (domain)	Orquestración única, parámetros, validaciones de entrada
Cubit + State (presentation)	Estados posibles (sealed class), transiciones, mensajes de error
Page (presentation)	Qué muestra cada estado, interacciones que disparan eventos
Supabase (backend)	Tablas, columnas, constraints, políticas RLS, triggers, RPC

1.8 Formato delta (ADDED / MODIFIED / REMOVED)

La spec de un cambio describe solo lo que cambia respecto a las specs archivadas:

- `## ADDED Requirements` — capacidad nueva
- `## MODIFIED Requirements` — requisito completo reescrito que reemplaza al anterior
- `## REMOVED Requirements` — qué desaparece y por qué

Esto mantiene las specs cortas y hace el diff auditable.

Salida de Fase 1: `proposal.md` + `specs/{capacidad}/spec.md`

■ Puerta 1 — Aprobación de requisitos

- [] Cada historia pasa la prueba de precisión
- [] Todos los requisitos usan notación EARS con ID único
- [] Los bordes están cubiertos (valores límite, vacío, negativos, expirados)
- [] Las preguntas abiertas del alcance están cerradas o documentadas
- [] El formato delta es correcto (ADDED/MODIFIED/REMOVED según toque)
- [] El Impact Report justifica el alcance elegido

Fase 2 · Diseño — CÓMO construirlo

Diluye **Mapeo a capas**, **Contratos**, **Flujo de datos** y parte de **Backend**.

2.1 Ficheros afectados (antes "Mapeo")

Tabla obligatoria de `design.md`. Sin ella, el agente improvisa rutas y rompe convenciones:

Elemento	Capa	Archivo	Regla asociada
CartItem	domain/entity	<code>lib/features/cart/domain/entities/cart_item.dart</code>	REQ-001
CartRepository	domain/repository	<code>lib/features/cart/domain/repositories/cart_repository.dart</code>	REQ-002..007
CartModel	data/model	<code>lib/features/cart/data/models/cart_model.dart</code>	REQ-001
CartRemoteDataSource	data/datasource	<code>lib/features/cart/data/datasources/cart_remote_data_source.dart</code>	REQ-002
AddItemToCart	domain/usecase	<code>lib/features/cart/domain/usecases/add_item_to_cart.dart</code>	REQ-003
CartCubit / CartState	presentation	<code>lib/features/cart/presentation/cubit/...</code>	REQ-008..010
CartPage	presentation	<code>lib/features/cart/presentation/pages/cart_page.dart</code>	REQ-011

2.2 Contratos Dart primero (antes "Contratos")

Escribe las interfaces ANTES de la implementación. Son la spec ejecutable del dominio:

```
abstract interface class CartRepository {
  Future<Either<Failure, Cart>> getCart();
  Future<Either<Failure, Unit>> addItem({required String productId, required int quantity});
  Future<Either<Failure, Unit>> removeItem({required String productId});
  Future<Either<Failure, Cart>> applyCoupon({required String code});
}
```

Estados UI como sealed class:

```
sealed class CartState {}
class CartInitial extends CartState {}
class CartLoading extends CartState {}
class CartLoaded extends CartState { final Cart cart; CartLoaded(this.cart); }
class CartError extends CartState { final String message; CartError(this.message); }
```

Si un contrato cambia respecto a una feature existente → requisito MODIFIED, no silencio.

2.3 Flujo de datos (antes "Flujo")

Documento ASCII del recorrido completo, incluidos caminos de error:

```
CartPage —onTap→ CartCubit.addItem(productId)
                    | emit CartLoading
                    ↓
                AddItemToCart(repository)
                    ↓
    CartRepositoryImpl → valida RN (stock, límite) local si aplica
                    ↓
    CartRemoteDataSource.addCartItem(...)
                    | supabase.from('cart_items').insert(...)
                    ↓
                Supabase: INSERT con RLS (auth.uid() = user_id)
                    ↓
    ← Either.right(Unit) ← mapeo ← respuesta
    |
    └─ Either.left(Failure) → CartError(message) → SnackBar
```

2.4 Backend Supabase (contrato de datos)

En `design.md` se fija ANTES de implementar:

- **Tablas:** `carts`, `cart_items`, `coupons` — columnas, tipos, FKs, constraints
- **RLS = tus antiguas RS:** cada política citada por su escenario (`auth.uid() = user_id`; admin solo lectura vía claim `role`)
- **Cálculos:** subtotal/impuesto en el cliente (entity pura) vs RPC atómico cuando hay concurrencia — decidir y anotar por qué
- **Migración:** número siguiente en `supabase/migrations/`, idempotente, nunca tocar migraciones previas

2.5 Decisiones explícitas + heurística sénior (antes ADRs ligeros)

Cada decisión no obvia se registra: qué se decide, alternativas descartadas, por qué. Señales de que una decisión merece registro: elegir entre dos patrones válidos, depender de un paquete nuevo, tocar seguridad, cambiar un contrato público.

Heurística sénior aplicada (qué haría un dev con años en este repo):

- Copia el patrón de la feature más parecida existente antes de inventar uno
- fpdart `Either` siempre; jamás exceptions cruzando capas de dominio
- Un UseCase = una operación; el Cubit orquesta varios
- Sin lógica de negocio en widgets ni en models

Salida de Fase 2: `design.md`

■ Puerta 2 — Aprobación del diseño

```
[ ] La tabla de ficheros afectados cubre TODOS los requisitos de la Puerta 1
[ ] Los contratos compilan mentalmente (firmas coherentes entre capas)
[ ] El flujo de datos cubre caminos de éxito Y de error
[ ] RLS especificada para toda tabla nueva/modificada
[ ] Ninguna decisión importante quedó implícita
[ ] Se respeta el patrón de las features existentes (o se justifica el cambio)
```

Fase 3 · Tareas — EN QUÉ ORDEN

Diluye **Descomposición en tareas** y **Estimación**.

3.1 Anatomía de tarea atómica

Cada tarea: 1–3 ficheros, verificable en aislado, con criterios de éxito objetivos.

```
- [ ] 2.1 Implementar CartModel
    Rol: experto en Flutter/Dart con Clean Architecture
    Tarea: serialización JSON→entity para carts y cart_items
    Restricciones: freezed opcional según pubspec; snake_case desde Supabase;
                  null-safety estricta; sin lógica de negocio
    Éxito: fromJson/toJson roundtrip; tests unitarios pasan
```

3.2 Oleadas estándar para una feature Clean Architecture

Dentro de una oleada las tareas son independientes (paralelizables); entre oleadas hay dependencia:

```
OLEADA 1 – Dominio (independientes entre sí)
  1.1 Entity Cart/CartItem (+ invariantes)
  1.2 Interface CartRepository
  1.3 UseCases (uno por operación)

OLEADA 2 – Capa de datos + Backend Supabase
  2.1 Migración SQL tablas + RLS           - si aplica; abre la capa: el flujo de datos es probable de inmediato
  2.2 Models (fromJson/toJson)
  2.3 RemoteDataSource
  2.4 CartRepositoryImpl

OLEADA 3 – Estado y presentación
  3.1 CartState (sealed)
  3.2 CartCubit
  3.3 CartPage + widgets

OLEADA 4 – Integración (cableado de la app)
  4.1 Registro en service_locator.dart
  4.2 Ruta en app_router.dart

OLEADA 5 – Tests
  5.1 Unit tests entidades/usecases
  5.2 Test repository impl (mock datasource) + integración contra la migración real
  5.3 Widget test página clave
```

3.3 Commits atómicos

1 tarea = 1 commit con mensaje convencional (`feat(cart): add CartModel with json mapping`). Beneficios: reversión granular, bisección eficaz, trazabilidad requisito→commit. Al cerrar la oleada: revisión contra la spec (no diff-and-hope).

3.4 Descomposición justa

Demasiado gruesa (>3 archivos, "implementar feature") = agente improvisa. Demasiado fina ("crear archivo", "añadir import") = ruido y pérdida de contexto. El punto dulce: una responsabilidad verificable.

Salida de Fase 3: `tasks.md`

■ Puerta 3 — Aprobación de tareas

```
[ ] Cada requisito de la spec tiene ≥1 tarea que lo implementa
[ ] Cada tarea referencia su requisito (trazabilidad bidireccional)
[ ] Las dependencias definen el orden de oleadas correctamente
[ ] Ninguna tarea excede ~3 archivos
[ ] Hay tareas de test para los escenarios críticos
```

Fase 4 · Implementación

Un único motor — `/opsx-apply-change` — según **quién escribe el código**. El **Modo** se declara por cambio en la primera línea de `tasks.md`:

Modo: andamiaje — scaffold IA + lógica crítica tuya

- `/opsx-apply-change` genera scaffold por tarea (entity/model/datasource/repo/usecases/cubit/state/pages) con cuerpos `UnimplementedError()` y comentarios TODO citando el requisito, deja la casilla en `- [ ]` y pausa
- Tú implementas las tareas siguiendo `tasks.md` oleada por oleada (la práctica de escribir la lógica crítica se preserva)
- Delegación automática: DI → skill `di-getit-scaffold`, routing → skill `go-route-scaffold`

Modo: completo — la IA escribe todo, tú verificas

- `/opsx-apply-change` — el agente lee `proposal/specs/design/tasks` y ejecuta todas las tareas, commit por tarea, escribiendo el 100% del código
- Al cierre de cada oleada auditas el resultado contra la spec
- Checklist completo de auditoría: `06-auditoria-codigo-ia.md`

¿andamiaje o completo? Matriz de decisión

Criterio	andamiaje (tú implementas la crítica)	completo (IA escribe todo)
Lógica crítica: dinero, permisos, estados	✔ entiendes cada línea que firmas	⚠ solo si los escenarios EARS son exhaustivos
CRUD, formularios, brownfield quirúrgico	posible overkill	✔ terreno ideal
Estás aprendiendo el stack o el dominio	✔ implementar ES aprender	✗ no aprendes nada
Specs con supuestos ocultos (falla el Clarity Gate)	✔ implementar te obliga a resolverlos	✗ el error se multiplica silenciosamente

Regla del libro: puedes delegar la *escritura*, nunca la aprobación de la spec (Puerta 1) ni la verificación final (Puerta 3).

Verificación contra spec

Al cierre de cada oleada:

Tarea	La spec dice	El código hace	Cumple
2.1 CartModel	roundtrip JSON completo	✔ roundtrip cubierto en test	✔
3.3 CartCubit	estados sealed + mensajes RN001/RN002	falta mensaje RN006	✗ → fix

Clarity Gate y cierre

- Clarity Gate:** ¿un agente distinto, con solo la spec, produciría código equivalente? Si dudas, la spec tiene supuestos ocultos → corrígela AHORA (sigue siendo barato).
- Archive:** `openspec archive <cambio>` mueve el change folder a specs consolidadas. Las specs vivas describen el sistema ACTUAL.

Boundaries del proyecto

Marco Always / Ask First / Never adaptado a Flutter + Supabase. Ajústalo a tu equipo y decláralo en la constitución del proyecto:

Nivel	En este stack
Always (sin preguntar)	Crear ficheros nuevos dentro de su feature; seguir patrón existente; añadir tests; formatear con dart format
Ask First	Cambiar contratos existentes (MODIFIED); añadir paquete a pubspec; crear tabla/modificar esquema; nueva ruta con guard
Never	Eliminar/reescribir migraciones previas; desactivar RLS; tocar secretos/.env; subir build.gradle/signing; borrar datos de producción

Proporcionalidad según complejidad

SDD es proporcional al riesgo. No todos los cambios necesitan las 3 puertas completas:

Nivel	Ejemplos	Proceso mínimo
Simple	Texto UI, color, constante, renombrado local	Proposal breve → implementar directo (sin design.md)
Intermedia	Nueva pantalla CRUD sobre patrón existente	Proposal + spec + design ligero + tasks → 1 puerta combinada
Compleja	Feature nueva multi-capa, pagos, cambios de contrato, seguridad	Flujo completo con 3 puertas

Regla práctica: si dudarías en mergearlo sin revisión de otro humano, es al menos Intermedia.

Specs vivas y trazabilidad

- La spec se actualiza **en el mismo commit** que el código que cambia el comportamiento (nunca después, nunca "luego").
- Matriz de trazabilidad mínima (vive en tasks.md): requisito ↔ tarea ↔ test ↔ commit.
- Tras archive, las capacidades archivadas son la documentación viva del sistema: léelas antes de planear el próximo cambio (alimentan el siguiente Impact Report).

REQ-003 (spec) → tarea 1.3 AddItemToCart (tasks) → test add_item_test.dart → commit feat(cart): ...

Errores comunes (anti-patrones aplicados)

Error	Síntoma	Prevención
Spec zombi	Código cambió, spec no	Actualizar spec en el mismo commit
Teatro de especificación	Docs perfectas, código que no las cumple	Puertas con checklist verificable, revisión por lotes
Sobreespecificación	Spec de 500 líneas para un botón	Proporcionalidad; EARS solo donde importa
Requisitos vagos	"debería funcionar bien"	Prueba de precisión + EARS obligatorio
Design omitido	Agente inventa carpetas y nombres	Tabla de ficheros afectados obligatoria
Tareas gigantes	"implementar feature completa"	Límite 1-3 ficheros por tarea
Delta mal usado	Reescribir specs enteras sin MODIFIED	Solo ADDED/MODIFIED/REMOVED según toque
Puertas saltadas	"ya lo apruebo mientras codeo"	El coste crece por fase; volver a la puerta más barata

Referencias

- Teoría: [01-teoria-sdd.md](#) → libro oficial en [pdf/](#)
- Herramienta CLI: [03-openspec-guia-practica.md](#)
- Plantilla lista para copiar: [04-plantilla-cambio-openspec.md](#)
- Cheat sheet: [05-referencia-rapida.md](#)
- Ejemplos completos: [ejemplos-cambios/](#)

03 - OpenSpec: Guía Práctica

La herramienta líder de SDD. Specs como markdown vivo en tu repositorio, ejecutables por 30+ agentes de IA.

Qué es OpenSpec

OpenSpec es un framework ligero de Spec Driven Development, open source (MIT), creado por Fission-AI. Tiene 65.7k estrellas en GitHub y es la herramienta más adoptada para implementar SDD en proyectos reales.

Filosofía:

- Fluid, no rígido
- Iterativo, no waterfall
- Fácil, no complejo
- Diseñado para brownfield (código existente), no solo greenfield
- Escalable de proyectos personales a empresas

No necesita: API keys, MCP servers, ni configuración compleja.

Instalación

```
# Requiere Node.js 20.19.0 o superior
npm install -g @fission-ai/openspec@latest
```

Verificar instalación:

```
openspec --version
```

Inicialización en un proyecto Flutter

```
# Navega a la raíz de tu proyecto Flutter
cd mi-proyecto-flutter

# Inicializa OpenSpec
openspec init
```

Qué crea:

```
mi-proyecto-flutter/
├── openspec/
│   ├── specs/           ← Specs vivas del proyecto
│   │   └── .gitkeep
│   ├── changes/        ← Cambios en progreso
│   │   └── .gitkeep
├── AGENTS.md           ← Constitución del proyecto (instrucciones para agentes IA)
└── .openspec/          ← Configuración interna
```

openspec init pregunta qué agentes usas y genera sus archivos de configuración. Con OpenCode seleccionado genera:

```
.opencode/commands/
├── opsx-explore.md      → invocable como /opsx-explore
├── opsx-propose.md      → /opsx-propose
├── opsx-new-change.md   → /opsx-new-change
└── ...                  (un archivo por comando)
```

Otros agentes reciben su equivalente (`CLAUDE.md` + `.claude/commands/opsx/`, `.cursorrules`, `.github/copilot-instructions.md`, etc.).

Verificado contra CLI v1.6.0 (@fission-ai/openspec). Si tu versión difiere, ejecuta `openspec --version` y `openspec update` tras instalar.

Workflow completo

Paso 1: Explorar antes de decidir

```
# En tu agente IA (OpenCode, Claude Code, Cursor, etc.)  
/opsx-explore
```

El agente lee tu codebase y te ayuda a pensar antes de escribir nada. Útil cuando no estás seguro de cómo implementar algo.

Ejemplo:

```
Tú: /opsx-explore  
IA: ¿Qué quieres explorar?  
Tú: Quiero agregar modo oscuro pero no estoy seguro de cómo hacerlo limpiamente  
IA: [lee tu setup de temas] La ruta más limpia: ThemeMode en el Cubit raíz +  
      ThemeData claro/oscuro en app_theme.dart, persistido en SharedPreferences.  
      Sin dependencias nuevas. ¿Lo acotamos?  
Tú: Sí, hagámoslo
```

Paso 2: Proponer un cambio

```
/opsx-propose add-dark-mode
```

OpenSpec genera una carpeta completa de cambio:

```
openspec/changes/add-dark-mode/  
├── proposal.md      ← Por qué y qué cambia  
├── specs/           ← Requirements y escenarios  
│   ├── theme/  
│   │   └── spec.md  
├── design.md        ← Decisiones técnicas  
└── tasks.md         ← Checklist de implementación
```

El agente genera esto automáticamente. Tú solo revisas y ajustas.

Paso 3: Revisar el plan

Antes de que el agente escriba código, revisas:

- `proposal.md` — ¿Estamos resolviendo el problema correcto?
- `specs/` — ¿Los requisitos son claros y verificables?
- `design.md` — ¿Las decisiones técnicas son coherentes con el codebase?
- `tasks.md` — ¿Las tareas están en el orden correcto?

Paso 4: Ejecutar

```
/opsx-apply-change
```

El agente implementa las tareas una por una:

```
✓ 1.1 ThemeCubit + estados sealed  
✓ 1.2 Persistencia de preferencia  
✓ 2.1 ThemeData claro/oscuro en app_theme.dart  
✓ 2.2 Ruta y wiring en app_router.dart
```

Paso 5: Archivar

```
/opsx-archive-change
```

El cambio se archiva y las specs se actualizan. Listo para el siguiente feature.

Qué son las specs en OpenSpec

Las specs son **markdown simple** con requisitos concretos y escenarios. No hay sintaxis especial que aprender.

Ejemplo de spec:

```
ADDED Requirements

### Requirement: Theme selection
The app SHALL let users switch between light and dark themes,
defaulting to the system preference.

#### Scenario: User toggles dark mode
- **WHEN** the user clicks the theme toggle
- **THEN** the app switches to dark mode and persists the choice

#### Scenario: System preference detection
- **GIVEN** the user has not set a theme preference
- **WHEN** the app loads
- **THEN** the theme matches the system setting
```

Cómo se organizan:

```
openspec/specs/
├── auth-login/
│   └── spec.md
├── auth-session/
│   └── spec.md
├── checkout-cart/
│   └── spec.md
└── checkout-payment/
    └── spec.md
```

Cada spec vive en su carpeta, al lado del código que implementa. Cuando un agente necesita contexto, lee la spec. Cuando alguien nuevo se une al equipo, browsa la biblioteca.

Slash commands disponibles (OpenCode, CLI v1.6.0)

Comando	Función
<code>/opsx-explore</code>	Explorar opciones antes de decidir (no crea cambio)
<code>/opsx-propose <nombre></code>	Crear propuesta + specs + design + tasks
<code>/opsx-new-change</code>	Nuevo cambio con workflow guiado completo
<code>/opsx-continue-change</code>	Continuar un cambio existente donde quedó
<code>/opsx-update-change</code>	Registrar progreso de tareas en el cambio
<code>/opsx-apply-change</code>	Ejecutar las tareas del cambio actual
<code>/opsx-verify-change</code>	Verificar que la implementación cumple las specs
<code>/opsx-archive-change</code>	Archivar cambio completado y consolidar specs
<code>/opsx-bulk-archive-change</code>	Archivar varios cambios completados
<code>/opsx-sync-specs</code>	Sincronizar deltas archivados con las specs vivas
<code>/opsx-onboard</code>	Onboarding de un nuevo desarrollador

Nota: el prefijo puede variar según el agente seleccionado en `init` / `update` (Claude Code usa `.claude/commands/opsx/<id>.md`, etc.). Los IDs de workflow son los mismos; verifica los archivos generados en tu proyecto.

Dos estilos de arranque: `/opsx-propose` es el **modo copiloto** (la IA redacta la primera versión de `proposal/spec/design/tasks` y tú apruebas). `/opsx-new-change` es el **modo artesano** (crea solo el esqueleto y lo llenas tú; la IA refina después con `/opsx-update-change`). Ambos convergen en la misma Puerta 1 — detalles en `02-sdd-flutter-supabase.md` §Dos modos de crear el cambio.

Comandos de la CLI

```
openspec init          # Inicializa OpenSpec en el proyecto (interactivo)
openspec update        # Regenera archivos de agentes tras actualizar la CLI
openspec list          # Lista cambios activos
openspec list --specs  # Lista specs archivadas
openspec view          # Dashboard interactivo de cambios y specs
openspec change <id>   # Muestra un cambio concreto
openspec archive <id>  # Archiva un cambio (equivale al slash command)
openspec spec <cap>    # Inspecciona una capacidad archivada
openspec validate      # Valida formato de changes/specs contra los schemas
openspec doctor        # Diagnóstico de la instalación local
```

La validación (`openspec validate`) comprueba que cada spec tenga secciones `WHY` / `Purpose` y requisitos en formato delta correcto — úsala antes de cada puerta.

Ejemplo completo: Login con Supabase Auth

Contexto

Proyecto Flutter con Clean Architecture, Supabase como backend.

Paso 1: Explorar

```
/opsx-explore
→ Quiero agregar login con Supabase Auth
→ El agente lee tu codebase y sugiere la mejor integración
```

Paso 2: Proponer

```
/opsx-propose add-supabase-auth
```

Lo que genera:

`openspec/changes/add-supabase-auth/proposal.md`:

```
# Proposal: Add Supabase Authentication

Problem
Users need to authenticate to access protected features.

Solution
Integrate Supabase Auth with email/password login, following Clean Architecture.

Impact
- New feature: `auth` under `lib/features/`
- Dependencies: supabase_flutter (already in pubspec)
- No breaking changes to existing features
```

`openspec/changes/add-supabase-auth/specs/auth-login/spec.md`:

ADDED Requirements

Requirement: Email/password login

The system SHALL authenticate users via email and password using Supabase Auth.

Scenario: Successful login

- GIVEN a registered user with valid credentials
- WHEN the user submits the login form
- THEN the system returns a User entity and navigates to the home screen

Scenario: Invalid credentials

- GIVEN a user with wrong password
- WHEN the user submits the login form
- THEN the system displays "Credenciales inválidas" and stays on the login screen

Scenario: Network error

- GIVEN no internet connection
- WHEN the user submits the login form
- THEN the system displays "Error de conexión" with a retry option

Requirement: Session persistence

The system SHALL persist the user session across app restarts.

Scenario: App restart with valid session

- GIVEN a user previously logged in
- WHEN the app restarts
- THEN the user is redirected to the home screen without re-authenticating

`openspec/changes/add-supabase-auth/design.md`:

Design: Supabase Auth

Architecture

Follow Clean Architecture layers:

- Domain: User entity, AuthRepository interface, LoginUseCase
- Data: AuthDataSource (Supabase), AuthRepositoryImpl, UserModel
- Presentation: AuthCubit, AuthState, LoginPage

Decisions

- Use Supabase GoTrue client for auth operations
- Store session via Supabase's built-in persistence
- Emit auth state changes via Stream in Cubit

`openspec/changes/add-supabase-auth/tasks.md`:

Tasks

- [] 1. Create User entity (domain/entities/user.dart)
- [] 2. Create AuthRepository interface (domain/repositories/auth_repository.dart)
- [] 3. Create LoginUseCase (domain/usecases/login.dart)
- [] 4. Create UserModel with snake_case mapping (data/models/user_model.dart)
- [] 5. Create AuthRemoteDataSource (data/datasources/auth_remote_datasource.dart)
- [] 6. Create AuthRepositoryImpl (data/repositories/auth_repository_impl.dart)
- [] 7. Create AuthCubit + AuthState (presentation/cubit/)
- [] 8. Create LoginPage with form pattern (presentation/pages/)
- [] 9. Register in DI (service_locator.dart)
- [] 10. Add route (app_router.dart)
- [] 11. Write unit tests for LoginUseCase
- [] 12. Write widget test for LoginPage

Paso 3: Revisar

Revisas cada artefacto. Ajustas si algo no está bien (ej: agregar "recordar sesión" al spec).

Paso 4: Ejecutar

/opsx-apply-change

- El agente ejecuta las 12 tareas una por una
- Cada tarea genera código que cumple la spec

Paso 5: Archivar

```
/opsx-archive-change
→ Las specs de auth-login se actualizan
→ El cambio queda documentado en el historial
```

Comparativa con otras herramientas

Característica	OpenSpec	Spec Kit (GitHub)	Kiro (AWS)
Tipo	CLI open source	CLI open source	IDE dedicado
Facilidad	Ligero, 1 comando	Más pesado, Python setup	Integrado, sin CLI
Flexibilidad	Fluido, sin fases rígidas	Fases rígidas	Flujo impuesto
Agentes	30+ soportados	~30 soportados	Solo Claude
Brownfield	Sí (diseñado para ello)	Sí	Sí
Specs en repo	Sí (openspec/)	Sí (.spec/)	No (en el IDE)
Coste	Gratis	Gratis	Requiere suscripción

Conclusión: OpenSpec es la opción más flexible y ligera. Spec Kit es más estructurado pero más pesado. Kiro es potente pero te encierra en su IDE.

Actualización

```
# Actualizar OpenSpec
npm install -g @fission-ai/openspec@latest

# Regenerar instrucciones de agentes en el proyecto
openspec update
```

Configuración de telemetría

OpenSpec recopila stats anónimas (solo comandos, no contenido). Para desactivar:

```
# Opción 1: config global
openspec config set telemetry.enabled false

# Opción 2: variable de entorno
export OPENSPEC_TELEMETRY=0
```

Errores comunes

Error	Causa	Solución
<code>command not found: openspec</code>	No está instalado globalmente	<code>npm install -g @fission-ai/openspec@latest</code>
<code>Node.js version too old</code>	Versión de Node menor a 20.19.0	Actualizar Node.js
Slash commands no aparecen	Agentes no configurados	Ejecutar <code>openspec update</code> en el proyecto
Specs no se generan	No se ejecutó <code>/opsx-propose</code> primero	Ejecutar propose antes de apply

Referencias

- **Sitio web:** openspec.dev
- **GitHub:** github.com/Fission-AI/OpenSpec

- **Discord:** discord.gg/YctCnvvshC
- **Metodología aplicada a tu stack:** [02-sdd-flutter-supabase.md](#)
- **Plantilla de cambio:** [04-plantilla-cambio-openspec.md](#)
- **Ejemplos listos para copiar:** [ejemplos-cambios/](#)

04 - Plantilla de Cambio OpenSpec

Copia esta estructura a `openspec/changes/<kebab-case-nombre>/` en tu proyecto Flutter y completa cada archivo. Cada sección indica qué fase del flujo produce (ver `02-sdd-flutter-supabase.md`).

Antes de cada puerta, ejecuta `openspec validate` para comprobar el formato.

Estructura de la carpeta

```
openspec/changes/add-<feature>/
├── proposal.md           ← Fases 0 + 1 (Impact Report + alcance)
├── specs/
│   └── <capacidad>/      ← una carpeta por capacidad afectada
│       └── spec.md       ← Fase 1 (requisitos EARS, formato delta)
├── design.md            ← Fase 2 (opcional: solo complejidad Intermedia+)
└── tasks.md             ← Fase 3
```

Convención de nombre: `add-` para features nuevas, `update-` para modificar existentes, `remove-` para retirarlas.

proposal.md

```
# Proposal: add-<feature>

Impacto (Impact Report)
<!-- 5-15 líneas. Obligatorio en brownfield. -->
- Features afectadas: `lib/features/...` — [cuáles y cómo]
- Reutilizable: [contratos/widgets/tablas que ya existen]
- Supabase: tablas `[...]`, RLS existente `[...]`, migración n° `<n>`
- DI / rutas: `service_locator.dart` (+<n> registros), `app_router.dart` (+1 ruta)
- Riesgos: [los 2-3 principales]

Why (Problema)
[2-4 frases: qué necesidad real existe hoy sin resolver]

What Changes (Solución)
[Qué se construye/cambia, en lenguaje del dominio]

Capabilities
### New Capabilities
- `<capacidad>`: [una frase]

### Modified Capabilities
- `<capacidad>`: [qué cambia y por qué] <!-- si aplica -->

Scope (Alcance)
**Incluye:**
- [...]
**No incluye:**
- [...] <!-- explícito: pagos, envíos, realtime... lo que NO toca -->
**Dependencias:** [...]
**Suposiciones:** [...]
**Preguntas abiertas:** [→ cerrar antes de Puerta 1]

Actores y permisos
| Actor | Puede | No puede | Mapeo RLS |
|-----|-----|-----|-----|
| Cliente | ... | ... | auth.uid() = user_id |

Impact
- Código: [ficheros nuevos/modificados estimados]
- Datos: [tablas nuevas/columnas/migraciones]
- Breaking changes: [ninguno / cuáles]
```

```
WHY
<!-- Por qué existe esta capacidad, ligado al problema del proposal -->

Purpose
[1-2 frases: qué garantiza esta capacidad]

ADDED Requirements

### Requirement: <Nombre del requisito>
<Enunciado EARS resumen: qué hará el sistema y para quién>

#### Scenario: <Camino feliz>
- **WHEN** <disparador>
- **THEN** el sistema DEBERÁ <comportamiento observable>
- **AND** <efecto secundario si aplica>

#### Scenario: <Caso borde>
- **IF** <condición límite>
- **THEN** el sistema DEBERÁ <respuesta controlada>

#### Scenario: <Error>
- **IF** <fallo esperable>
- **THEN** el sistema DEBERÁ mostrar "<mensaje exacto>" y no alterar el estado

<!-- Repetir Requirement por cada operación atómica.
      Cubrir SIEMPRE: camino feliz + bordes + errores + seguridad (RLS). -->

MODIFIED Requirements
<!-- Solo si este cambio modifica requisitos ya archivados.
      Se reescribe el requisito COMPLETO; reemplaza al anterior. -->

REMOVED Requirements
<!-- Solo si algo deja de existir. Explicar el porqué. -->
```

Reglas EARS rápidas:

- **Ubicuo**: "El sistema DEBERÁ..." — invariable
- **Evento**: "CUANDO X EL sistema DEBERÁ..." — interacción
- **Estado**: "MIENTRAS X EL sistema DEBERÁ..."
- **No deseado**: "SI X ENTONCES el sistema DEBERÁ..."
- **Opcional**: "DONDE X EL sistema DEBERÁ..."

design.md (opcional — Complejidad Simple lo omite)

```
# Design: add-<feature>
```

Context

```
[Estado actual relevante: patrón existente que se sigue, restricciones descubiertas]
```

Goals / Non-Goals

- Goals: [...]
- Non-Goals: [...]

Decisions

```
<!-- Una por decisión no obvia -->
```

```
### D1: <título>
```

- Decisión: [...]
- Alternativas descartadas: [...]
- Por qué: [...]

Ficheros afectados

```
| Elemento | Capa | Archivo | Req |
|-----|-----|-----|-----|
| <Entity> | domain/entity | lib/features/x/domain/entities/... | REQ-00x |
| <Repository> | domain/repository | lib/features/x/domain/repositories/... | ... |
| <Model> | data/model | lib/features/x/data/models/... | ... |
| <DataSource> | data/datasource | lib/features/x/data/datasources/... | ... |
| <UseCase> | domain/usecase | lib/features/x/domain/usecases/... | ... |
| <Cubit>/<State> | presentation/cubit | lib/features/x/presentation/cubit/... | ... |
| <Page> | presentation/pages | lib/features/x/presentation/pages/... | ... |
| Registro DI | core/di | lib/core/di/service_locator.dart | ... |
| Ruta | core/router | lib/core/router/app_router.dart | ... |
```

Contratos Dart clave

```
abstract interface class <X>Repository {
    Future<Either<Failure, T>> operacion(...);
}
sealed class XState {}
class XInitial extends XState {}
class XLoading extends XState {}
class XLoaded extends XState { final T data; XLoaded(this.data); }
class XError extends XState { final String message; XError(this.message); }
```

Flujo de datos

```
Page → Cubit → UseCase → RepositoryImpl → DataSource → Supabase
                                     |
                             Either.left(Failure) ←
    ← CartError(message) ← mapeo de failures
```

Backend Supabase

- Tablas: [columnas, tipos, FKs, constraints]
- RLS: [política por escenario de seguridad]
- RPC/Triggers: [si hay cálculo atómico o concurrencia]
- Migración: supabase/migrations/NNNN_*.sql (idempotente, no editar previas)

Boundaries aplicables a este cambio

[Qué NO debe hacer el agente aquí: ej. no tocar feature Y, no añadir paquetes]

tasks.md

```
# Tasks: add-<feature>
> Modo de implementación: andamiaje
> andamiaje = la IA genera el scaffold (throw UnimplementedError() + TODOs citando REQ) y TÚ implementas los métodos críticos
> completo = la IA escribe todo (100%) y tú auditas cada diff contra la spec

1. Dominio
- [ ] 1.1 Crear <Entity> con invariantes
    Rol: experto Flutter/Dart + Clean Architecture
    Éxito: invariantes cubiertas por test unitario
    Req: REQ-001 · Commit: feat(x): add Entity

- [ ] 1.2 Definir interface <X>Repository (Either<Failure,T>)
    Éxito: firmas coinciden con design.md
    Req: REQ-002..007 · Commit: feat(x): add repository contract

- [ ] 1.3 UseCases (uno por operación)
    Éxito: mensajes exactos de los escenarios en failures de dominio
    Req: REQ spec · Commit: feat(x): add usecases

2. Capa de datos + Backend Supabase
- [ ] 2.1 Migración SQL + políticas RLS ← si aplica; abre la capa para probar el flujo de datos
    (+ RPC si hay concurrencia · Edge Functions si aplica)
    Éxito: openspec validate + supabase db reset OK
    Req: escenarios RS · Commit: db(x): add tables and rls
- [ ] 2.2 <Model> fromJson/toJson (snake_case ↔ camelCase)
- [ ] 2.3 <RemoteDataSource> (llamadas exactas + errores de red)
- [ ] 2.4 <RepositoryImpl> (mapeo excepciones → Failure)

3. Estado y presentación
- [ ] 3.1 <State> sealed class
- [ ] 3.2 <Cubit> (transiciones + mensajes exactos de los escenarios)
- [ ] 3.3 <Page> + widgets (un render por estado)

4. Integración
- [ ] 4.1 Registros en service_locator.dart
- [ ] 4.2 Ruta en app_router.dart

5. Tests
- [ ] 5.1 Unit tests entidades/usecases (bordes de la spec)
- [ ] 5.2 Test repository impl (mock datasource) + integración contra la migración real
- [ ] 5.3 Widget test página clave

Trazabilidad
| Req | Tarea(s) | Test | Commits |
|-----|-----|-----|-----|
| REQ-001 | 1.1 | entity_test | abc123 |
```

Configuración — que `/opsx-apply-change` respete el modo

El modo se declara **por cambio** en la primera línea de `tasks.md` (véase arriba). Para que el agente de `/opsx-apply-change` —el único motor de implementación— lo cumpla de forma fiable, añade una vez al `openspec/config.yaml` de tu proyecto:

```
# openspec/config.yaml — se añade una sola vez; el interruptor por cambio es la línea "Modo" de tasks.md
operations:
  apply:
    guidance:
      - "Si tasks.md declara 'Modo: andamiaje': NO escribas los bodies. Por cada tarea genera solo el scaffolding (throw UnimplementedError() + TODO citando REQ-xxx), NO marques la casilla - [x], y pausa para que el desarrollador complete e implemente."
      - "Si tasks.md declara 'Modo: completo': implementa la tarea al 100% contra la spec (camino de éxito y error de los escenarios) y márcala - [x]."
```

Por qué funciona: ese archivo vive en tu proyecto (`openspec/config.yaml`, junto a `lib/`), no es parte de la herramienta. El CLI lo lee en tiempo de ejecución (`openspec instructions apply`) y lo inyecta en el prompt del agente como `operationGuidance`, que el flujo built-in ordena leer y seguir. El interruptor por cambio sigue siendo la línea `Modo` de `tasks.md` — el `config.yaml` se configura una sola vez.

Comportamiento según el modo (vía `/opsx-apply-change`):

Tarea	andamiaje	completo
Cuerpos	la IA deja <code>throw UnimplementedError()</code> + TODO citando REQ	la IA escribe el código completo
Casilla	se deja en <code>- []</code> (la tarea NO está terminada)	se marca <code>- [x]</code> al cumplir la spec
Final	el agente pausa y pasas a implementar	el agente termina y tú auditas cada diff

La skill `clean-arch-feature` queda como **herramienta manual opcional (fuera del flujo)**: con `andamiaje` genera el scaffold del cambio completo y deja los bodies para ti; con `completo` puedes pedirle que implemente. En el flujo, el grano del andamiaje lo decide el modo dentro de `/opsx-apply-change`: `andamiaje` = scaffold por tarea y pausa en cada una; `completo` = escribe el cambio entero.

Checklist de las 3 puertas

- Puerta 1 (tras Fase 1):** historias precisas · EARS con IDs · bordes cubiertos · alcance cerrado · delta correcto.
- Puerta 2 (tras Fase 2):** ficheros afectados completos · contratos coherentes · flujo con caminos de error · RLS especificada · decisiones registradas.
- Puerta 3 (tras Fase 3):** todo requisito tiene ≥1 tarea · trazabilidad bidireccional · tareas ≤3 ficheros · orden de oleadas correcto · tests presentes · `tasks.md` declara su modo (`andamiaje` | `completo`).

Cierre

```
openspec validate          # formato OK
# ... implementar (Fase 4) ...
openspec archive add-<feature>
```

Ejemplos reales completados: `ejemplos-cambios/`

05 - Referencia Rápida

Cheat sheet de SDD: fases, puertas, EARS, boundaries, proporcionalidad y OpenSpec.

Las 4 fases de SDD

REQUISITOS →	DISEÑO →	TAREAS →	IMPLEMENTACIÓN
¿QUÉ?	¿CÓMO?	¿EN QUÉ ORDEN?	¿EJECUTAR (agente/humano)

Fase	Entregable	Quién participa
Requisitos	Historias, criterios, reglas	PO + equipo
Diseño	Arquitectura, contratos, tablas	Developer + agente
Tareas	Unidades atómicas con prompts	Developer
Implementación	Código, tests, commits	Agente + developer

Las 3 puertas de aprobación

PUERTA 1	PUERTA 2	PUERTA 3
Requisitos → Diseño	Diseño → Tareas	Tareas → Implementación

Puerta	Pregunta clave	Qué verificar
1	¿Resolvemos el problema correcto?	Requisitos completos, criterios verificables, reglas explícitas
2	¿Es viable y coherente?	Patrones del codebase, dependencias, orden lógico
3	¿Estas tareas producen el diseño?	Cobertura, sin huecos, prompts precisos

Regla de oro: El coste de corregir un error crece exponencialmente por fase. Corregir en requisitos es trivial; corregir en implementación es caro.

Notación EARS (5 patrones)

Patrón	Palabra clave	Template	Ejemplo Flutter
Ubicuo	(sin condición)	"El sistema SHALL [comportamiento]"	"La app SHALL mostrar el nombre del usuario en todas las páginas"
Evento	CUANDO	"CUANDO [evento], el sistema SHALL [respuesta]"	"CUANDO el usuario pulsa logout, el sistema SHALL cerrar sesión"
Estado	MIENTRAS	"MIENTRAS [condición], el sistema SHALL [comportamiento]"	"MIENTRAS el formulario sea inválido, el botón SHALL estar deshabilitado"
No deseado	SI	"SI [error], el sistema SHALL [recuperación]"	"SI la sesión expira, el sistema SHALL redirigir al login"
Opcional	DONDE	"DONDE [configuración], el sistema SHALL [comportamiento]"	"DONDE el usuario habilitó biometría, el sistema SHALL ofrecer login por huella"

Template universal:

MIENTRAS [precondición], CUANDO [evento], el sistema debe [respuesta]

Sistema de boundaries (Always / Ask First / Never)

Nivel	Significado	Ejemplo Flutter
Always	El agente ejecuta sin preguntar	<code>flutter analyze</code> , naming conventions, const constructors
Ask First	Requiere aprobación antes de ejecutar	Añadir dependencias, modificar migrations, cambiar routing
Never	Líneas rojas absolutas	Commitear .env, editar build.gradle, modificar RLS de producción

Regla de evolución: A medida que el equipo gana confianza, algo que hoy es Ask First puede pasar a Always.

Principio de proporcionalidad

Pregunta de calibración

¿Qué pasa si el agente toma la decisión equivocada en este punto?

Respuesta	Acción
"Lo corrijo en 2 minutos"	No necesita especificarse
"Pierdo horas o introduzco un bug sutil"	Sí necesita especificarse

Cuándo usar SDD vs vibe coding

Situación	Herramienta
Bug trivial, script, prototipo	Vibe coding
Feature compleja, cambio con riesgo, trabajo en equipo	SDD
Feature nueva en codebase existente	SDD + Impact Report
Refactor grande	SDD (flujo completo)

Equivalencias de terminología (glosario de dilución)

Equivalencia de términos entre el método de diseño anterior y SDD:

Equivalencia	Ahora (SDD estándar)	Dónde vive
Paso Alcance	Sección Scope del <code>proposal.md</code>	Fase 1
Formular + Actorizar	Historias precisas + tabla de actores/permisos	Fase 1
Descomponer	Operaciones atómicas → requisitos dirigidos por evento EARS	Fase 1
Entidades	Specs por componente Clean Arch	Fase 1
Reglas RN (negocio)	Requisitos EARS (ubicuo/evento/no deseado) + escenarios	<code>specs/*/spec.md</code>
Reglas RT (técnicas)	Decisiones explícitas en <code>design.md</code>	Fase 2
Reglas RS (seguridad)	Escenarios EARS + políticas RLS	spec + design \$Backend
Mapeo a capas	Tabla "Ficheros afectados"	<code>design.md</code>
Contratos + ADRs	Sección Contratos Dart + Decisions	<code>design.md</code>
Flujo de datos	Diagrama Page→Cubit→UseCase→Repo→DataSource→Supabase	<code>design.md</code>

Equivalencia	Ahora (SDD estándar)	Dónde vive
Criterios BDD	Escenarios GIVEN-WHEN-THEN dentro de cada Requirement	deltas

Nota: Los "Criterios BDD" y los "escenarios EARS" son la misma cosa: cada `#### Scenario:` con `GIVEN/WHEN/THEN` en `spec.md` es el criterio de aceptación. Ver también sección 1.5 de `02-sdd-flutter-supabase.md`. | Trazabilidad | Matriz Req ↔ tarea ↔ test ↔ commit | `tasks.md` | | Estimación de complejidad | Proporcionalidad Simple/Intermedia/Compleja | todo el flujo | | Sprint Planning (Scrum) | Puerta 3: aprobar `tasks.md` antes de ejecutar | ceremonias | | Sprint Review | Revisión final contra la spec al cierre de la implementación | ceremonias |

Maldición de las instrucciones

A más instrucciones en un prompt, menor probabilidad de que el modelo cumpla cada una.

Señales de spec sobrecargada:

- El agente ignora restricciones escritas
- Cumple lo del principio pero no lo del final
- Mejora cuando se reduce la spec
- Acierta con tareas aisladas y falla con varias juntas

Solución: Dividir, no acumular. Cinco specs de 10 instrucciones > una de 50.

Specs vivas: 3 niveles de vida

Nivel	Descripción	Deriva
Spec-first	Se escribe antes, guía la tarea, se descarta	No aplica
Spec-anchored	Se mantiene como documentación viva	Riesgo real → actualizar en mismo commit
Spec-as-source	La spec es el artefacto principal; código se regenera	Desaparece por diseño (experimental)

Clarity Gate (prueba de calidad)

¿Puede un agente diferente generar código equivalente solo con la spec?

- **Sí** → spec clara y completa
- **No** → faltan supuestos implícitos → actualizar spec

OpenSpec: comandos rápidos (CLI v1.6.0, OpenCode)

Comando	Acción
<code>npm install -g @fission-ai/openspec@latest</code>	Instalar
<code>openspec init</code>	Inicializar en el proyecto
<code>openspec validate</code>	Validar formato de changes/specs
<code>/opsx-explore</code>	Explorar opciones
<code>/opsx-propose <nombre></code>	Crear cambio completo
<code>/opsx-apply-change</code>	Ejecutar tareas
<code>/opsx-verify-change</code>	Verificar cumplimiento de specs
<code>/opsx-archive-change</code>	Archivar cambio
<code>openspec update</code>	Regenerar instrucciones de agentes

Plantilla de spec (copia y pega)

```
### Requirement: [Nombre] [Componente]
The [Sistema/Capa] SHALL [comportamiento].

#### Scenario: [Éxito]
- GIVEN [precondición]
- WHEN [acción]
- THEN [resultado esperado]

#### Scenario: [Error]
- GIVEN [condición de error]
- WHEN [acción]
- THEN [resultado de error]
```

Antipatrones de SDD

Antipatrones	Descripción	Solución
Sobreespecificación	Más tiempo en specs que en código	Principio de proporcionalidad
Teatro de especificación	Specs que nadie revisa de verdad	Puertas de aprobación reales
Documentación zombi	Specs que nadie consulta	decidir qué mantener, qué dejar morir
Rigidez excesiva	Mismo proceso para bug trivial y feature compleja	Calibrar intensidad
Puertas como cuellos de botella	Aprobación bloqueada por ausencia	Delegación asíncrona

Métricas (cap. 22 del libro)

Tipo	Ejemplos
De flujo	Tiempo de ciclo por fase, % de cambios que pasan la puerta a la primera, retrabajos por fase
De resultado	Bugs en producción, deriva spec-código, velocidad de onboarding

Qué NO medir: líneas de spec producidas, número de requisitos — fomenta teatro de especificación.

Referencia completa

Documento	Ubicación
Teoría: mapa del libro oficial	01-teoria-sdd.md · pdf/SDDEquiposAgiles_v.1.pdf
Resumen state-of-the-art	pdf/Guia-SDD-equipos-agiles.pdf
Metodología aplicada a Flutter+Supabase	02-sdd-flutter-supabase.md
OpenSpec guía práctica	03-openspec-guia-practica.md
Plantilla de cambio	04-plantilla-cambio-openspec.md
Ejemplos completos	ejemplos-cambios/

06 · Auditoría de código generado por IA (Modo completo)

Escenario: la IA escribe **todo** el código y tú solo verificas. Este documento es el manual del nuevo rol: **de autor a auditor**. Complementa Fase 4 de 02-sdd-flutter-supabase.md y se aplica con los ejemplos de `ejemplos-cambios/`.

El cambio de rol

En el Modo `andamiaje` implementas y entiendes cada línea porque la escribiste. En el Modo `completo` tu valor ya no está en escribir sino en **dos momentos que no se delegan**:

Momento	Qué haces	Por qué no se delega
Puerta 1	Apruebas proposal + spec EARS + design	La spec es el contrato; una spec ambigua produce código incorrecto <i>con total confianza</i>
Puerta 3	Auditas el código contra la spec	La IA optimiza para "que compile y pase"; tú vela por "que cumpla el contrato"

Todo lo demás (escribir, tests, commits, refactors) lo ejecuta el agente con `/opsx-apply-change`.

El ciclo de auditoría por oleada

```
Agente: ejecuta oleada N (commit atómico por tarea)
↓
Tú: audita el diff de la oleada contra specs/*/spec.md + design.md → este documento
↓
├ Cumple → siguiente oleada
└ No cumple → pide fix citando REQ + escenario exacto
  ├── El código estaba mal → fix puntual
  └ La spec era ambigua → corrige la spec PRIMERO (/opsx-update-change),
    luego re-aplica. (Clarity Gate: sigue siendo barato aquí)
```

Regla práctica: **nunca aceptes "ya funciona" sin el requisito citado**. Cada línea de código debe poder responder "¿qué REQ y qué escenario me obligan a existir?".

Checklist de auditoría

Úsalo al cierre de cada oleada. Los ejemplos citan el cambio `add-cart` como referencia de cómo debería verse.

1 · Contratos intactos

- ☐ Las firmas del repository interface coinciden **verbatim** con `design.md` \$Contratos Dart clave (nombres, tipos, named params). Ejemplo: si design dice `Future<Either<Failure, Cart>> getCart()`, no existe un `Cart?` ni un `Future<Cart>` "por simplicidad"
- ☐ Todo método de repository/usecase retorna `Either<Failure, T>` — cero excepciones lanzadas hacia arriba, cero `return null`
- ☐ Cero `try/catch` que traga errores (catch vacío o que loguea y continúa); los errores se convierten en `Failure` en el datasource/repository impl, nunca antes
- ☐ No se inventaron métodos fuera del alcance: si `design.md` \$Ficheros afectados no lista `deleteCart()`, ese método no debe existir
- ☐ Los archivos creados corresponden 1:1 con la tabla de ficheros afectados (rutas literales)

2 · Sealed states exhaustivos y mensajes exactos

- ☐ El state sealed tiene todas sus subclases construidas en el cubit y consumidas en la UI. Ejemplo: `sealed class CartState` con `Initial/Loading/Loaded/Error` — ningún `switch` con caso `_` que oculte estados olvidados

- ☐ Cada transición visible en el flujo de datos de design.md tiene su `emit`. Ejemplo: `addItem` emite `CartLoading` → luego `CartLoaded(cart recalculado)` o `CartError(...)`
- ☐ **Los mensajes de error son los literales exactos de los escenarios EARS**, no paráfrasis. Ejemplos reales de add-cart:
 - `"Producto {nombre} sin stock disponible"` (REQ-001) — no vale `"Stock insuficiente"`
 - `"Has alcanzado el límite de 50 productos"` (REQ-001) — el 50 viene de la regla, no está hardcoded en la UI
 - `"La cantidad debe ser mayor a 0"` (REQ-001)
 - `"El cupón {codigo} ha expirado"` (REQ-004)
 - `"El descuento supera el límite permitido"` (REQ-004, RN tope 50%)
- ☐ Los mensajes nacen de domain (usecase/failure), no se componen strings en la UI

3 • Reglas en la capa correcta

- ☐ Las reglas de negocio (RN) viven en **domain**: entity o usecases. Si encuentras `if (descuento > subtotal * 0.5)` dentro del cubit o de un widget → mal ubicada (en add-cart ese tope se evalúa al aplicar cupón, en `ApplyCoupon`)
- ☐ Sin duplicación inconsistente cliente/servidor: si la validación de stock vive en el RPC `add_cart_item` (Decisión D3 de design.md), NO debe existir además un `if (product.stock <= 0)` en Dart que pueda divergir. Puede haber guardas de UX, pero la fuente de verdad es una sola
- ☐ Los cálculos económicos están donde dice design.md. Ejemplo: Decisión D1 pone subtotal/impuesto/total como métodos puros de la entity `Cart` — verificar que no terminaron en el datasource
- ☐ Decisiones D1–D4 (o las que haya) respetadas; cada archivo clave puede citar su decisión en un comentario

4 • Supabase y seguridad ⚠ (la categoría más crítica)

- ☐ RLS habilitada en TODAS las tablas nuevas de la migración (`alter table ... enable row level security`)
- ☐ Políticas por tabla × actor × operación, y coinciden con design.md §Backend Supabase. Ejemplo add-cart: `USING (auth.uid() = user_id)` para `carts`; acceso a `cart_items` vía join con el carrito propio
- ☐ RPCs `security definer` revisados línea por línea: validaciones internas (stock, límites), transacción atómica, y que NO bypassen RLS más de lo declarado
- ☐ Migración idempotente si design lo declara (add-cart: `create policy if not exists`)
- ☐ Cero `service_role` key en código cliente; cero `.rpc()` que escriba tablas sensibles sin validar ownership interno
- ☐ Columnas de la migración = columnas declaradas en design.md §Backend (ni faltantes ni extra)
- ☐ El datasource usa exactamente las tablas/RPCs del design (ej.: `supabase.rpc('add_cart_item', params: {...})`) y no queries ad-hoc inventadas

5 • Tests contra la spec

- ☐ Existe **al menos un test por Scenario** de cada Requirement ADDED (los escenarios son la especificación de pruebas)
- ☐ Los nombres de test citan el REQ: `test('REQ-001: producto sin stock muestra mensaje y no modifica carrito')`
- ☐ Los asserts usan los literales exactos de los mensajes (si cambias el texto en la spec, el test debe romper)
- ☐ Tests de regla: mutar la entrada viola la salida esperada. Ejemplo: aplicar cupón con descuento > 50% del subtotal DEBE retornar Failure con el mensaje del escenario
- ☐ Cero tests siempre-verdes (assert trivial, mock que devuelve lo mismo que se pasa)
- ☐ Test de roundtrip JSON para cada model (add-cart: `CartModel.fromJson(toJson)` preserva montos)

6 • Trazabilidad completa

- ☐ Matriz Req ↔ tarea ↔ test de tasks.md llena, sin huecos
- ☐ Cero `UnimplementedError()` sobrevivientes (o los que queden tienen tarea abierta explícita)
- ☐ Los TODOs generados citan el REQ correcto (un TODO de REQ-003 dentro de `applyCoupon` es señal de copia-pegar)
- ☐ Commits atómicos: un commit por tarea, mensaje convencional referenciando la tarea (`feat(cart): RPC add_cart_item (task 2.3, REQ-001)`)

7 • Red flags generales

- ☐ El diff no toca archivos fuera de la tabla de ficheros afectados (scope creep silencioso)

- ☐ No hay dependencias nuevas en pubspec.yaml sin justificación en design.md (add-cart declara "no añadir paquetes")
- ☐ Sin over-engineering: abstracciones, interfaces o genéricos que nadie pidió
- ☐ Sin código muerto, imports sin usar, ni comentarios que expliquen decisiones NO tomadas
- ☐ Los boundaries del change folder (§Boundaries en design.md) se respetan. Ejemplo: "No desactivar RLS bajo ninguna circunstancia"

Ejemplo de auditoría real (REQ-001, escenario "Producto sin stock")

- Spec dice** (spec.md): *"IF el stock del producto es 0 THEN mostrar 'Producto {nombre} sin stock disponible' sin modificar el carrito"*
- Design dice**: la validación es del RPC `add_cart_item` dentro de una transacción (D3)
- Auditar**:
 - ☐ El SQL del RPC valida stock y hace `raise exception` con ese mensaje exacto
 - ☐ El failure llega como `Either.left(Failure("Producto {nombre} sin stock disponible"))`, no como excepción
 - ☐ El cubit lo emite como `CartError(mensaje)` y la UI lo muestra tal cual
 - ☐ Hay test que inserta producto con stock 0 y afirma el mensaje + que `cart_items` no cambió
 - ☐ La transacción revierte si algo falla a mitad (atomicidad de D3)

Si las cinco casillas pasan, ese escenario está auditado. Repite por escenario; un Requirement con 4 escenarios son 4 mini-auditorías.

Señales de que NO deberías estar en Modo completo

Sé honesto contigo mismo:

- Leíste el checklist y más de la mitad de los términos te suenan vagos (`Either`, sealed, RLS, security definer...)
- No sabrías distinguir un `try/catch` tragador de un manejo legítimo de errores
- Firmarías el diff "porque compila y los tests pasan"

→ Cambia a **Modo andamiaje** (implementas tú sobre scaffold) hasta que el checklist sea lectura natural. Ahí es exactamente donde entra `trabajar-sin-ia/`: criterio de auditoría se construye escribiendo código, no leyendo diffs.

Resumen en una línea

La IA puede escribir todo el código, pero el contrato (spec) lo apruebas tú al inicio y el cumplimiento (auditoría) lo certificas tú al final. Entre esos dos momentos, delega sin culpa.

Siguiente paso: aplica este checklist sobre `ejemplos-cambios/add-cart/` simulando que otro agente lo implementó — es el mejor entrenamiento antes de usarlo en producción.

Ejemplos de Cambios OpenSpec

6 cambios completos listos para estudiar o copiar a `openspec/changes/` en tu proyecto. Derivan de los casos del módulo histórico `02-DISEÑO-FEATURE` (conservado como referencia).

El patrón que repiten los 6 cambios

Todos siguen el mismo esqueleto. Léelo una vez y no tendrás que deducirlo de cada ejemplo:

Paso	Qué produces	Comando	Tu papel	Validación
0 · Clasificar	Simple / Intermedia / Compleja	—	tú decides	proporcionalidad
1 · Crear cambio	carpeta <code>openspec/changes/<feat>/</code>	<code>/opsx-propose add-nombre</code>	revisas el esqueleto	<code>openspec validate</code>
2 · proposal.md	WHY + Impact + Scope + Actores	<code>openspec instructions proposal --change <n></code>	IA redacta → tú apruebas	—
3 · spec.md	requisitos EARS con escenarios	<code>openspec instructions spec --change <n></code>	IA redacta → tú apruebas	Puerta 1 + Clarity Gate
4 · design.md	ficheros, contratos, backend, decisiones	<code>openspec instructions design --change <n></code>	IA redacta → tú apruebas	Puerta 2
5 · tasks.md	5 oleadas + trazabilidad + Modo	<code>openspec instructions tasks --change <n></code>	IA redacta → tú apruebas	validate
6 · Implementar	código + tests	<code>/opsx-apply-change</code>	andamiaje: implementas la lógica crítica · completo: auditas cada diff	tests verdes
7 · Verificar · archivar	specs vivas consolidadas	<code>/opsx-verify-change</code> → <code>/opsx-archive-change</code>	auditas	Puerta 3

Observación clave: hasta el Paso 5 la IA redacta los artefactos y tú los apruebas. Solo el Paso 6 depende del **Modo** declarado en `tasks.md`: `andamiaje` = la IA deja scaffold por tarea y tú implementas la lógica crítica; `completo` = la IA escribe todo y tú auditas cada diff contra la spec. El motor es siempre el mismo: `/opsx-apply-change`. Las plantillas anotadas (qué va en cada parte) están en `07-guia-paso-a-paso.md` y la plantilla completa en `04-plantilla-cambio-openspec.md`.

Carpeta	Complejidad	Qué demuestra
<code>add-cart/</code>	Intermedia	Walkthrough completo puerta por puerta; reglas económicas y cupones
<code>approve-reservas/</code>	Compleja	Multi-actor, concurrencia (RPC atómico), realtime, cron
<code>add-elearning-progress/</code>	Intermedia→Compleja	Gating por RLS, dos capacidades en un cambio
<code>add-facturacion/</code>	Compleja	Máquina de estados regulada, numeración atómica
<code>add-delivery-seguimiento/</code>	Compleja	Realtime GPS (Broadcast), geolocalización PostGIS
<code>add-buyers/</code>	Simple→Intermedia	Cambio pequeño con una puerta combinada

Cómo usarlos

- Aprender:** lee `add-cart/` completo junto a `02-sdd-flutter-supabase.md` — cada archivo corresponde a una fase
- Copiar:** clona la carpeta más parecida a tu feature en `openspec/changes/<tu-feature>/` y adapta
- Calibrar:** compara `add-buyers/` (ligero) con `approve-reservas/` (completo) para aplicar la proporcionalidad

Proposal: add-cart

Ejemplo walkthrough completo del flujo SDD. Deriva de la práctica "Carrito de Compras" del módulo histórico (ver trabajar-sin-ia).

Impacto (Impact Report)

- Features afectadas: ninguna directamente (feature nueva); `products` existente provee catálogo
- Reutilizable: patrón CRUD de features existentes, `Failure` hierarchy en core
- Supabase: tablas nuevas `carts`, `cart_items`, `coupons`; RLS por dueño; migración nº 0007
- DI / rutas: `service_locator.dart` (+8 registros), `app_router.dart` (+1 ruta `/cart` con guard de sesión)
- Riesgos: concurrencia al actualizar stock; cálculo de impuestos duplicado cliente/servidor

Why (Problema)

Los clientes no pueden acumular productos antes de pagar: cada compra es de un solo artículo. Se pierde ticket promedio y no hay lugar donde aplicar descuentos.

What Changes (Solución)

Carrito de compras por cliente autenticado: agregar/quitar productos, actualizar cantidades, ver resumen económico (subtotal, impuesto, total) y aplicar cupones de descuento.

Capabilities

New Capabilities

- `shopping-cart`: gestión del carrito con validación de stock, reglas económicas y cupones

Scope (Alcance)

Incluye:

- Agregar/quitar productos y actualizar cantidades
- Ver resumen con subtotal, impuesto (16%) y total
- Aplicar cupones de descuento (porcentaje o monto fijo)

No incluye:

- Procesar pagos
- Envíos y seguimiento
- Programa de fidelización
- Carritos anónimos (requiere sesión iniciada)

Dependencias: catálogo de productos (stock), autenticación **Suposiciones:** un carrito pertenece a un solo cliente; precio unitario se congela al agregar **Preguntas abiertas:** ~~cupones combinables?~~ → NO (decisión D2); ~~límite de items?~~ → RN001 = 50

Actores y permisos

Actor	Puede	No puede	Mapeo RLS
Cliente	CRUD items propios, aplicar cupón, ver su resumen	Ver/modificar carritos ajenos	<code>auth.uid() = user_id</code>
Admin	Ver carritos abandonados, ajustar precios	Modificar carrito ajeno	claim <code>role = 'admin'</code> solo lectura
Inventario	Validar stock vía RPC	Escribir en carts	función security definir

Impact

- Código: ~14 ficheros nuevos en `lib/features/cart/`
- Datos: 3 tablas + 1 RPC + políticas RLS + migración 0007
- Breaking changes: ninguno

Spec: shopping-cart

WHY

Los clientes necesitan acumular productos y ver el coste real antes de pagar. Sin carrito, cada compra es unitaria y no existen descuentos.

Purpose

Garantizar un carrito por cliente autenticado con integridad económica (precios congelados, impuesto 16%, cupones validados) y aislamiento total entre clientes (RLS).

ADDED Requirements

Requirement: Agregar producto al carrito (REQ-001)

El sistema agregará productos al carrito del cliente validando stock y límites.

Scenario: Producto con stock disponible

- **WHEN** el cliente agrega un producto con stock > 0
- **THEN** el sistema DEBERÁ agregarlo con cantidad inicial 1 y congelar su precio unitario
- **AND** recalcular subtotal, impuesto y total

Scenario: Producto sin stock

- **IF** el stock del producto es 0
- **THEN** el sistema DEBERÁ mostrar "Producto {nombre} sin stock disponible" sin modificar el carrito

Scenario: Límite de items alcanzado

- **IF** el carrito ya contiene 50 productos distintos
- **THEN** el sistema DEBERÁ mostrar "Has alcanzado el límite de 50 productos"

Scenario: Cantidad inválida al actualizar

- **IF** la cantidad indicada es ≤ 0
- **THEN** el sistema DEBERÁ mostrar "La cantidad debe ser mayor a 0"

Requirement: Quitar producto del carrito (REQ-002)

El cliente eliminará items de su carrito.

Scenario: Item existente

- **WHEN** el cliente quita un item del carrito
- **THEN** el sistema DEBERÁ eliminarlo y recalcular los totales

Requirement: Ver resumen económico (REQ-003)

El sistema calculará subtotal, impuesto y total.

Scenario: Carrito con items

- **MIENTRAS** el carrito tenga items
- **EL SISTEMA DEBERÁ** mostrar $\text{subtotal} = \sum(\text{precioUnitario} \times \text{cantidad})$, $\text{impuesto} = 16\% \times (\text{subtotal} - \text{descuento})$ y $\text{total} = \text{subtotal} - \text{descuento} + \text{impuesto}$

Scenario: Carrito vacío

- **MIENTRAS** el carrito no tenga items
- **EL SISTEMA DEBERÁ** mostrar estado vacío sin totales calculados

Requirement: Aplicar cupón de descuento (REQ-004)

El cliente aplicará un cupón válido a su carrito.

Scenario: Cupón vigente aplicable

- **WHEN** el cliente aplica un cupón no expirado con usos disponibles
- **THEN** el sistema DEBERÁ recalcular el descuento según tipo (porcentaje o monto fijo)

Scenario: Cupón expirado

- **IF** la fechaExpiracion del cupón ya pasó
- **THEN** el sistema DEBERÁ mostrar "El cupón {codigo} ha expirado"

Scenario: Descuento sobre límite

- **IF** el descuento resultante supera el 50% del subtotal
- **THEN** el sistema DEBERÁ mostrar "El descuento supera el límite permitido"

Requirement: Aislamiento entre carritos (REQ-005)

Cada cliente accederá únicamente a su propio carrito.

Scenario: Lectura propia

- **GIVEN** un cliente autenticado
- **WHEN** consulta su carrito
- **THEN** solo recibe filas donde `auth.uid() = user_id`

Scenario: Intento de acceso ajeno

- **IF** un cliente intenta leer/escribir el carrito de otro usuario
- **THEN** Supabase DEBERÁ devolver conjunto vacío/denegación vía RLS

Requirement: Estados UI del carrito (REQ-006)

La interfaz reflejará cada estado del flujo.

Scenario: Carga inicial

- **WHEN** la página del carrito se abre
- **THEN** se muestra `CartLoading` hasta recibir datos

Scenario: Error de red

- **SI** falla la comunicación con Supabase
- **ENTONCES** se muestra "Error de conexión" con opción de reintentar

Design: add-cart

Context

Feature nueva siguiendo el patrón estándar del curso (Clean Architecture + Cubit + fpdart). Catálogo `products` ya existe con columna `stock`. El cálculo económico vive en la entity (puro, testeable); la validación de stock se delega a RPC para evitar carreras.

Goals / Non-Goals

- Goals: carrito íntegro por cliente; reglas económicas centralizadas en domain; cupones con límite de uso
- Non-Goals: pagos, envíos, fidelización, realtime (el resumen se refresca al entrar)

Decisions

D1: Cálculos económicos en la entity (cliente)

- Decisión: subtotal/impuesto/total como métodos puros de `Cart`
- Alternativas: RPC `calculate_cart_totals` en cada lectura
- Por qué: testable sin red, cero latencia extra; los montos se revalidan al crear la orden (fuera de alcance)

D2: Cupones NO combinables

- Decisión: un solo cupón activo por carrito
- Por qué: simplifica RN006 (tope 50%) y el modelo de datos; extensible después vía MODIFIED

D3: Stock validado por RPC atómico

- Decisión: `rpc.add_cart_item(p_product_id, p_qty)` valida stock y descuenta dentro de una transacción security definir
- Alternativas: SELECT+UPDATE desde el datasource
- Por qué: evita oversell por concurrencia (riesgo detectado en Impact Report)

D4: Precio congelado en cart_items

- Decisión: columna `unit_price` copiada al insertar
- Por qué: REQ-001 (RN007); el precio de catálogo puede cambiar después

Ficheros afectados

Elemento	Capa	Archivo	Req
Cart, CartItem, Coupon	domain/entity	lib/features/cart/domain/entities/{cart, cart_item, coupon}.dart	REQ-003
CartRepository	domain/repository	lib/features/cart/domain/repositories/cart_repository.dart	REQ-001..005
CartModel, CouponModel	data/model	lib/features/cart/data/models/	REQ-001..004
CartRemoteDataSource	data/datasource	lib/features/cart/data/datasources/cart_remote_data_source.dart	REQ-001..005
CartRepositoryImpl	data/repositories	lib/features/cart/data/repositories/cart_repository_impl.dart	REQ-001..005
AddItemToCart, RemoveItem, GetCart, ApplyCoupon	domain/usecase	lib/features/cart/domain/usecases/	REQ-001..004
CartCubit / CartState	presentation/cubit	lib/features/cart/presentation/cubit/	REQ-006

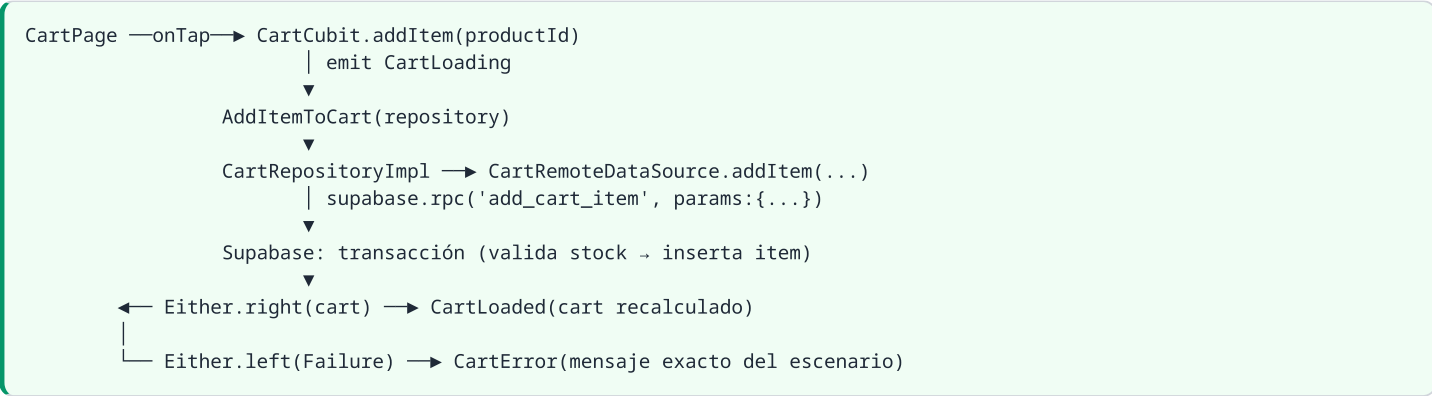
Elemento	Capa	Archivo	Req
CartPage + widgets	presentation/pages	lib/features/cart/presentation/pages/cart_page.dart	REQ-006
Registro DI	core/di	lib/core/di/service_locator.dart	—
Ruta /cart	core/router	lib/core/router/app_router.dart	—
Migración 0007	supabase/migrations	supabase/migrations/0007_carts_rls.sql	REQ-005

Contratos Dart clave

```
abstract interface class CartRepository {
  Future<Either<Failure, Cart>> getCart();
  Future<Either<Failure, Unit>> addItem({required String productId});
  Future<Either<Failure, Unit>> removeItem({required String productId});
  Future<Either<Failure, Unit>> updateQuantity({required String productId, required int quantity});
  Future<Either<Failure, Cart>> applyCoupon({required String code});
}

sealed class CartState {}
class CartInitial extends CartState {}
class CartLoading extends CartState {}
class CartLoaded extends CartState { final Cart cart; const CartLoaded(this.cart); }
class CartError extends CartState { final String message; const CartError(this.message); }
```

Flujo de datos



Backend Supabase

- Tablas:
 - `cards(id uuid pk, user_id uuid unique-auth.users, coupon_code fk nullable)`
 - `cart_items(id pk, cart_id fk, product_id fk, quantity int check >0, unit_price numeric, created_at)`
 - `coupons(code pk, type enum('percent','fixed'), value numeric, expires_at, max_uses, used_count)`
- RPC: `add_cart_item` security definer (valida stock + RN001 + RN003 atómicamente)
- RLS: enable en las 3 tablas; `USING (auth.uid() = user_id)` para carts y cart_items (via join); admin lectura con claim role
- Migración: `0007_carts_rls.sql`, idempotente (`create policy if not exists`)

Boundaries aplicables a este cambio

- No tocar features existentes ni sus contratos
- No añadir paquetes a pubspec (todo existe)
- No desactivar RLS bajo ninguna circunstancia

Tasks: add-cart

1. Dominio

- ☐ 1.1 Crear entities Cart, CartItem, Coupon con cálculos puros (subtotal, impuesto 16%, descuento tope 50%, total) Rol: experto Flutter/Dart + Clean Architecture Éxito: RN004/RN006 cubiertos por unit test de invariantes Req: REQ-003 · Commit: `feat(cart): add cart entities with pricing rules`
- ☐ 1.2 Definir interface CartRepository (Either<Failure,T>, 5 métodos) Éxito: firmas idénticas a design.md · Compila en main sin impl Req: REQ-001..005 · Commit: `feat(cart): add repository contract`
- ☐ 1.3 UseCases: GetCart, AddItemToCart, RemoveItem, UpdateQuantity, ApplyCoupon Restricciones: un UseCase = una operación; validaciones de entrada aquí Éxito: mensajes exactos de los escenarios (RN001-RN006) en failures de dominio Req: REQ-001..004 · Commit: `feat(cart): add usecases`

2. Capa de datos + Backend Supabase

- ☐ 2.1 Migración 0007_carts_rls.sql (3 tablas + RPC add_cart_item + RLS) Restricciones: idempotente; no editar migraciones previas; RLS enable SIEMPRE Éxito: `supabase db reset` OK; políticas citan escenarios REQ-005 Req: REQ-005 · Commit: `db(cart): add carts tables, rpc and rls`
- ☐ 2.2 CartModel/CouponModel (snake_case ↔ camelCase, roundtrip fromJson/toJson) Éxito: roundtrip cubierto en test Req: REQ-001..004 · Commit: `feat(cart): add data models`
- ☐ 2.3 CartRemoteDataSource (supabase.rpc / .from, manejo AuthException/SocketException) Éxito: cada método lanza excepciones mapeables a Failure Req: REQ-001..005 · Commit: `feat(cart): add remote data source`
- ☐ 2.4 CartRepositoryImpl (mapeo excepciones → Failure, delega datasource) Éxito: test con mock datasource pasa Req: REQ-001..005 · Commit: `feat(cart): implement repository`

3. Estado y presentación

- ☐ 3.1 CartState sealed class (Initial/Loading/Loaded/Error) Commit: `feat(cart): add cart state`
- ☐ 3.2 CartCubit (transiciones + mensajes EXACTOS de los escenarios) Éxito: cada escenario REQ-001..006 tiene su transición probada Req: REQ-006 · Commit: `feat(cart): add cart cubit`
- ☐ 3.3 CartPage + widgets (un render por estado; SnackBar para errores) Éxito: widget test renderiza Loaded y Error correctamente Req: REQ-006 · Commit: `feat(cart): add cart page`

4. Integración

- ☐ 4.1 Registros en service_locator.dart (datasource, repo, usecases ×5, cubit — lazy singletons/factory según patrón) Commit: `chore(di): register cart dependencies`
- ☐ 4.2 Ruta /cart en app_router.dart con guard de sesión Commit: `feat(router): add cart route`

5. Tests

- ☐ 5.1 Unit tests entities (bordes: stock 0, cantidad ≤0, cupón expirado, descuento >50%)
- ☐ 5.2 Test repository impl (mocks datasource; éxito + failure)
- ☐ 5.3 Widget test CartPage (estados Loading/Loaded/Error/vacío)

Trazabilidad

Req	Tarea(s)	Test	Cubre escenario
REQ-001	1.1, 2.1, 2.3, 1.3	entity_test	feliz/sin stock/límite/cantidad

Req	Tarea(s)	Test	Cubre escenario
REQ-002	1.2, 1.3	repo_test	quitar item
REQ-003	1.1	entity_test	resumen y vacío
REQ-004	1.3, 3.2	cubit_test	cupón vigente/expirado/tope
REQ-005	2.1	(política RLS)	aislamiento
REQ-006	3.1, 3.2, 3.3	widget_test	estados UI

Proposal: approve-reservas

Deriva del caso completo "Sistema de Reservas (Clínica Veterinaria)". Ejemplo de **complejidad Compleja**: multi-actor, concurrencia (doble reserva), realtime.

Impacto (Impact Report)

- Features afectadas: ninguna directa; requiere catálogo de doctores/mascotas
- Reutilizable: patrón CRUD estándar; `Failure` hierarchy
- Supabase: tabla nueva `appointments`; RLS vía join pets→owners; RPC atómico para slots
- DI / rutas: +2 cubits, +3 páginas, +3 registros ruta (agenda dueño / agenda doctor / agendar)
- Riesgos: doble reserva por concurrencia; huso horario de slots

Why (Problema)

Los dueños agendan por teléfono o van sin cita: colas, ausencias y agenda desordenada para los doctores.

What Changes (Solución)

Agenda de citas veterinarias: dueños agendan/cancelan/reagendan; doctores gestionan su día (completada/no-show); admins configuran horarios y bloqueos; recordatorio automático 24h antes.

Capabilities

New Capabilities

- `appointments`: agendamiento validado contra disponibilidad, con transiciones de estado auditables

Scope (Alcance)

Incluye: agendar, cancelar (≥2h antes), reagendar, ver agendas por actor, marcar completada/no-show, horarios laborales, bloqueo de días, recordatorio 24h. **No incluye:** facturación, recetas, pagos, historias clínicas. **Dependencias:** autenticación, catálogo doctores/mascotas/tipos de consulta. **Suposiciones:** duración según tipo de consulta; huso horario de la clínica. **Preguntas abiertas:** ¿reagendar conserva recordatorio? → Sí, se reprogra (D2).

Actores y permisos

Actor	Puede	No puede	Mapeo RLS
Dueño	Agendar/cancelar/reagendar citas de SUS mascotas	Ver citas ajenas	<code>pet_id in (select id from pets where owner_id = auth.uid())</code>
Doctor	Ver SU agenda, completar/cancelar/no-show	Editar datos del dueño	<code>doctor_id = auth.uid()</code>
Admin	Horarios, bloqueos, doctores	Agendar por un dueño	claim <code>role='admin'</code>
Notificaciones	Enviar recordatorio 24h	Cambiar citas	service role

Impact

- Código: ~20 ficheros en `lib/features/appointments/`
- Datos: tablas `appointments`, `work_schedules`, `blocked_days`, `reminders`; RPC atómico; cron pg_cron para no-show/recordatorios
- Breaking changes: ninguno

Spec: appointments

WHY

Sin agenda digital hay colas, ausencias y doctores sin visibilidad de su día.

Purpose

Garantizar citas únicas por slot (sin dobles reservas), con reglas de antelación y aislamiento estricto entre dueños.

ADDED Requirements

Requirement: Agendar cita (REQ-001)

El sistema creará citas validando disponibilidad, antelación y límites.

Scenario: Slot libre con antelación suficiente

- **WHEN** el dueño agenda un slot libre con ≥ 4 h de antelación
- **THEN** la cita DEBERÁ quedar `scheduled` con duración según tipo de consulta

Scenario: Doble reserva concurrente

- **IF** dos dueños agendan el mismo slot simultáneamente
- **THEN** el RPC DEBERÁ aceptar solo el primero y rechazar al segundo con "slot_ocupado"

Scenario: Antelación insuficiente

- **IF** faltan menos de 4 horas para el slot
- **THEN** el sistema DEBERÁ mostrar "Debes agendar con al menos 4 horas de antelación"

Scenario: Límite diario del doctor

- **IF** el doctor ya tiene 8 citas ese día
- **THEN** el sistema DEBERÁ mostrar "El doctor alcanzó su límite de citas diarias"

Scenario: Dos consultas el mismo día

- **IF** la mascota ya tiene una cita ese día
- **THEN** el sistema DEBERÁ mostrar "Ya existe una consulta para esta mascota ese día"

Requirement: Cancelar cita (REQ-002)

Scenario: Cancelación con antelación

- **WHEN** el dueño cancela con ≥ 2 h de antelación
- **THEN** la cita pasa a `cancelled`, se libera el slot y se notifica al doctor

Scenario: Fuera de plazo

- **IF** faltan menos de 2 horas
- **THEN** el sistema DEBERÁ mostrar "Solo puedes cancelar hasta 2 horas antes"

Requirement: Gestión del día del doctor (REQ-003)

Scenario: Marcar completada / no-show

- **WHEN** el doctor marca la cita como `completed` o `no_show` (pasados 15 min)
- **THEN** la transición DEBERÁ registrarse con timestamp del actor

Scenario: Agenda en vivo

- **MIENTRAS** el doctor tiene la agenda abierta
- **EL SISTEMA DEBERÁ** reflejar nuevas citas/cancelaciones vía realtime

Requirement: Aislamiento de datos (REQ-004)

Scenario: Dueño consulta sus citas

- **GIVEN** un dueño autenticado
- **WHEN** lista sus citas
- **THEN** solo recibe las de mascotas donde `owner_id = auth.uid()` (RLS)

Requirement: Recordatorio 24h (REQ-005)

Scenario: Recordatorio enviado

- **CUANDO** falten 24h para una cita `scheduled`
- **EL SISTEMA DEBERÁ** enviar notificación al dueño y marcar el recordatorio como enviado

Design: approve-reservas

Context

Multi-actor con concurrencia real (doble reserva) y realtime. Sigue el patrón estándar del curso.

Goals / Non-Goals

- Goals: cero dobles reservas; agenda en vivo; reglas de antelación centralizadas
- Non-Goals: pagos, recetas, historias clínicas

Decisions

D1: Disponibilidad validada en servidor (RPC atómico)

- Decisión: `rpc.agendar_cita(...)` valida slot + RN002/RN003/RN008 dentro de transacción; constraint único `(doctor_id, date_time)` de respaldo
- Alternativas descartadas: validación solo frontend (race condition); bloqueo optimista por versión (complejidad innecesaria)
- Por qué: la fuente de verdad del slot es la BD

D2: Reagendar = cancelar + crear atómicamente

- Decisión: RPC `reagendar_cita` hace ambas operaciones en una transacción y reprograma el recordatorio
- Por qué: evita estados intermedios inconsistentes

D3: Realtime para agendas

- Decisión: `supabase.from('appointments').stream()` filtrado por doctor/dueño
- Por qué: REQ-003 agenda en vivo sin polling

Ficheros afectados (resumen)

Elemento	Capa	Archivo
Appointment, Pet, WorkSchedule...	domain/entities	lib/features/appointments/domain/entities/
AppointmentRepository, VeterinarianRepository	domain/repositories	.../domain/repositories/
CreateAppointment, CancelAppointment, Reschedule, GetDoctorAgenda, GetOwnerAppointments, MarkCompleted, MarkNoShow	domain/usecases	.../domain/usecases/
Models + RemoteDataSource + LocalDataSource(caché) + Impls	data	lib/features/appointments/data/
AgendaCubit, CreateAppointmentCubit (+states)	presentation/cubit	.../presentation/cubit/
AgendaPage, CreateAppointmentPage, DetailPage	presentation/pages	.../presentation/pages/
Migraciones + RPC + RLS + pg_cron	supabase/migrations	0008_appointments_rls.sql

Contratos Dart clave

```
abstract interface class AppointmentRepository {
  Future<Either<Failure, Appointment>> create(CreateAppointmentParams params);
  Future<Either<Failure, Appointment>> cancel(String appointmentId);
  Future<Either<Failure, Appointment>> reschedule(String id, DateTime newDateTime);
  Future<Either<Failure, Appointment>> markCompleted(String appointmentId);
  Future<Either<Failure, Appointment>> markNoShow(String appointmentId);
  Future<Either<Failure, List<Appointment>>> getDoctorAgenda(String doctorId, DateTime date);
  Future<Either<Failure, List<Appointment>>> getOwnerAppointments(String ownerId, {AppointmentStatus? status});
}

sealed class AgendaState {}
class AgendaLoading extends AgendaState {}
class AgendaLoaded extends AgendaState { final List<Appointment> appointments; const AgendaLoaded(this.appointments); }
class AgendaError extends AgendaState { final String message; const AgendaError(this.message); }
```

Flujo de datos (agendar)

```
CreateAppointmentPage → CreateAppointmentCubit.submit()
    | emit Creating
    ▼
CreateAppointment → AppointmentRepository.create()
    ▼
RemoteDataSource.agendar(rpc) → Supabase transacción:
    valida RN001/RN002/RN003/RN008 → INSERT → commit
    ▼
Either.right(Appointment) → Created → navegar a detalle
Either.left(Failure("slot_ocupado")) → Error(mensaje) → refrescar slots
```

Backend Supabase

- `appointments(id, pet_id→pets, doctor_id→doctors, date_time timestamptz, duration_min, status enum(scheduled,completed,cancelled,no_show), created_at)`
- RLS: dueño vía subquery pets; doctor por `doctor_id = auth.uid()`
- RPC: `agendar_cita`, `reagendar_cita` (security definer, transaccionales)
- Cron: recordatorios 24h; no-show automático a los 15 min

Boundaries

No tocar catálogo de mascotas/doctores existente; no añadir paquetes; RLS nunca desactivada.

Tasks: approve-reservas

1. Dominio

- ☐ 1.1 Entities: Appointment (+status enum), WorkSchedule, BlockedDay
- ☐ 1.2 Interfaces AppointmentRepository + VeterinarianRepository (Either)
- ☐ 1.3 UseCases ×7 (uno por operación)

2. Capa de datos + Backend Supabase

- ☐ 2.1 Migración 0008: tablas + RPCs atómicos + RLS + pg_cron Éxito: doble reserva rechazada en SQL puro; políticas por actor
- ☐ 2.2 Models con roundtrip JSON (timestampz ↔ DateTime UTC) Restricción: slots siempre en huso horario de la clínica (RT001)
- ☐ 2.3 RemoteDataSource (RPC + select) y LocalDataSource (caché de agenda)
- ☐ 2.4 RepositoryImpls (mapeo excepciones → Failure con mensajes exactos)

3. Estado y presentación

- ☐ 3.1 AgendaState/CreateAppointmentState sealed + Cubits Éxito: stream realtime actualiza AgendaLoaded
- ☐ 3.2 Pages + widgets (slot selector, tarjeta cita, calendario)

4. Integración

- ☐ 4.1 DI + rutas con guards por rol (dueño/doctor/admin)

5. Tests

- ☐ 5.1 Unit: reglas RN001-RN008 como failures de dominio
- ☐ 5.2 Integration SQL: doble reserva, cancelación fuera de plazo
- ☐ 5.3 Widget: agenda loaded/error, flujo agendar feliz

Trazabilidad

Req	Tarea(s)	Test
REQ-001	1.3, 2.1	unit+SQL (doble reserva, antelación, límites)
REQ-002	1.3, 2.4	unit (2h antelación)
REQ-003	3.1, 3.2	cubit (transiciones + realtime)
REQ-004	2.1	integration RLS
REQ-005	2.1 (cron)	integration pg_cron

Proposal: add-elearning-progress

Deriva del caso "Plataforma de Cursos Online". Ejemplo Intermedia→Compleja: dos capacidades (catálogo/inscripción + progreso).

Impacto (Impact Report)

- Features afectadas: ninguna; feature nueva `elearning`
- Reutilizable: patrón CRUD, auth con claims de rol
- Supabase: tablas `courses`, `modules`, `lessons`, `enrollments`, `lesson_progress`; RLS por inscripción
- DI / rutas: +3 cubits (catálogo, player, instructor), +4 páginas
- Riesgos: fuga de contenido no inscrito (RN002); cálculo de progreso inconsistente

Why (Problema)

No existe forma de distribuir contenido educativo estructurado ni medir avance de estudiantes.

What Changes (Solución)

Catálogo con búsqueda backend, inscripción (gratuita/paga), lecciones gated por inscripción, progreso calculado y auto-completado del curso, flujo editorial borrador→revisión→publicado.

Capabilities

New Capabilities

- `course-catalog`: exploración y administración de cursos
- `enrollment-progress`: inscripción, gating de contenido y progreso

Scope (Alcance)

Incluye: catálogo paginado con búsqueda full-text backend, CRUD instructor, aprobación admin, inscribirse, ver/marcar lecciones, progreso %, auto-estado completada. **No incluye:** pagos reales (solo flag pagoId), certificados, comentarios. **Dependencias:** autenticación con roles. **Suposiciones:** videos alojados externamente (Vimeo/YouTube). **Preguntas abiertas:** ¿reinscripción tras completar? → NO en v1.

Actores y permisos

Actor	Puede	No puede	Mapeo RLS
Estudiante	Explorar, inscribirse, leer lecciones inscritas, SU progreso	Ver lecciones sin inscripción	progreso: <code>auth.uid() = student_id</code>
Instructor	CRUD SUS cursos/lecciones, reportes	Publicar (solo solicitar)	<code>instructor_id = auth.uid()</code>
Admin	Aprobar/rechazar publicación	Editar contenido ajeno	claim role
Pagos (ext.)	Confirmar pago	—	webhook

Impact

- Código: ~18 ficheros en `lib/features/elearning/`
- Datos: 5 tablas; RPC `enroll` atómico; vista `course_progress`
- Breaking changes: ninguno

Spec: enrollment-progress

WHY

El estudiante necesita acceso controlado al contenido y visibilidad de su avance.

Purpose

Gating estricto de lecciones por inscripción y progreso calculado de forma única y consistente.

ADDED Requirements

Requirement: Inscribirse a un curso (REQ-001)

Scenario: Primera inscripción

- **WHEN** el estudiante se inscribe a un curso publicado
- **THEN** se crea inscripción **activa** (y requiere pagoId si el curso es pago — RN007)

Scenario: Ya inscrito

- **IF** existe inscripción activa previa
- **THEN** el sistema DEBERÁ mostrar "Ya estás inscrito en este curso" (RN001)

Scenario: Auto-inscripción del instructor

- **IF** el instructor intenta inscribirse a su propio curso
- **THEN** el sistema DEBERÁ rechazarlo (RN006)

Requirement: Gating de contenido (REQ-002)

Scenario: Lección con inscripción activa

- **MIENTRAS** la inscripción esté **activa**
- **EL SISTEMA DEBERÁ** permitir leer/ver las lecciones del curso

Scenario: Sin inscripción

- **IF** no hay inscripción activa
- **THEN** el sistema DEBERÁ ocultar el contenido y ofrecer el botón de inscripción (RN002, RLS)

Requirement: Progreso (REQ-003)

Scenario: Marcar lección completada

- **WHEN** el estudiante marca una lección como completada
- **THEN** el progreso DEBERÁ recalcularse = $(\text{completadas} / \text{total}) \times 100$ (RN004)

Scenario: Curso completado

- **WHEN** todas las lecciones quedan completadas
- **THEN** la inscripción DEBERÁ pasar a estado **completada** (RN005)

Scenario: Progreso privado

- **ENTONCES** cada estudiante SOLO lee su propio progreso (RS001)

Requirement: Publicación editorial (REQ-004)

Scenario: Solicitar publicación sin lecciones

- **IF** el curso no tiene ≥ 1 lección
- **THEN** el sistema DEBERÁ mostrar "El curso necesita al menos una lección" (RN003)

Scenario: Aprobación admin

- **WHEN** el admin aprueba

- **THEN** el curso pasa a `publicado`; solo el claim role='admin' puede hacerlo (RN008)

Scenario: Eliminar con estudiantes

- **IF** hay inscripciones activas
- **THEN** solo se permite archivar, no eliminar (RN009)

Design: add-elearning-progress

Context

Dos capacidades en un feature folder. Búsqueda delegada al backend (RT001: full-text search con `websearch_to_tsquery`).

Goals / Non-Goals

- Goals: gating hermético; progreso consistente (una sola fórmula)
- Non-Goals: pasarela de pagos real, certificados

Decisions

D1: Progreso calculado en SQL (vista) y espejado en entity

- Decisión: vista `course_progress` (SQL puro, fuente de verdad); la entity expone el mismo cálculo para UI optimista
- Por qué: RN004 vive en UN lugar; la UI no espera roundtrip para pintar %

D2: Gating por RLS + validación de dominio

- Decisión: policy de lectura de lecciones exige inscripción activa; además el UseCase valida antes de navegar
- Por qué: defensa en profundidad — la RLS es la fuente de verdad (RN002)

D3: RPC `enroll` atómico

- Decisión: valida RN001/RN006/RN007 dentro de una transacción
- Por qué: evita inscripciones duplicadas por doble tap

Ficheros afectados (resumen)

Elemento	Capa	Archivo
Course, Module, Lesson, Enrollment, LessonProgress	domain/entities	lib/features/elearning/domain/entities/
CourseRepository, EnrollmentRepository	domain/repositories	...
GetCatalog, GetCourseDetail, Enroll, MarkLessonCompleted, RequestPublication, ApproveCourse...	domain/usecases	...
Models, RemoteDataSource, Impls	data	...
CatalogCubit, PlayerCubit (+states), InstructorCubit	presentation/cubit	...
CatalogPage, CourseDetailPage, LessonPlayerPage, MyCoursesPage	presentation/pages	...
Migración 0009 (tablas+RLS+vista+RPC)	supabase/migrations	...

Contratos Dart clave

```
abstract interface class EnrollmentRepository {
  Future<Either<Failure, Enrollment>> enroll({required String courseId});
  Future<Either<Failure, Unit>> markLessonCompleted({required String lessonId});
  Future<Either<Failure, CourseProgress>> getProgress({required String courseId});
}

sealed class PlayerState {}
class PlayerReady extends PlayerState { final Lesson lesson; final int progressPercent; const PlayerReady(this.lesson, this.progressPercent); }
class PlayerLocked extends PlayerState { final String message; const PlayerLocked(this.message); }
```


Flujo de datos (marcar completada)

```
LessonPlayerPage → PlayerCubit.markCompleted()
  ▼
MarkLessonCompleted → EnrollmentRepositoryImpl → DataSource
  | upsert lesson_progress (RS001 vía RLS)
  ▼
vista course_progress recalcula → Either.right(progress)
  ▼
PlayerReady(lesson, progress%) → si 100% → inscripción 'completada' (trigger)
```

Backend Supabase

- Tablas: courses(+status enum), modules(orden), lessons(tipo, contenidoUrl check tipo='video'→url no nula), enrollments(estado), lesson_progress(unique student+lesson)
- Vista: course_progress(student_id, course_id, percent)
- RLS: lecciones legibles solo con enrollment activa; progreso owner-only; instructor edita solo lo suyo (RS002)
- Trigger: al cubrir 100% → enrollments.estado='completada'

Boundaries

No integrar pasarela de pagos real en este cambio (MODIFIED futuro); no filtrar catálogo en cliente.

Tasks: add-elearning-progress

1. Dominio

- ☐ 1.1 Entities: Course, Module, Lesson, Enrollment, LessonProgress Éxito: RN004 implementado como método puro con test de borde (0 lecciones)
- ☐ 1.2 Interfaces CourseRepository + EnrollmentRepository
- ☐ 1.3 UseCases ×8

2. Capa de datos + Backend Supabase

- ☐ 2.1 Migración 0009: tablas + checks (RN010) + vista progreso + RPC enroll + RLS Éxito: SELECT de lección sin inscripción devuelve vacío (test SQL)
- ☐ 2.2 Models roundtrip (enums de estado incluidos)
- ☐ 2.3 RemoteDataSource (búsqueda full-text vía rpc; stream no requerido)
- ☐ 2.4 RepositoryImpls

3. Estado y presentación

- ☐ 3.1 CatalogCubit/PlayerCubit/InstructorCubit + states sealed Éxito: PlayerLocked cuando no hay inscripción (mensaje exacto)
- ☐ 3.2 CatalogPage (paginación), CourseDetailPage, LessonPlayerPage, MyCoursesPage

4. Integración

- ☐ 4.1 DI + rutas con guard por rol

5. Tests

- ☐ 5.1 Unit: RN001/RN004/RN006 como failures; fórmula progreso bordes
- ☐ 5.2 SQL: gating RLS, trigger completada, RPC duplicados
- ☐ 5.3 Widget: player locked/ready, catálogo paginado

Trazabilidad

Req	Tarea(s)	Test
REQ-001	1.3, 2.1	unit+SQL (duplicado, auto-inscripción)
REQ-002	1.3, 2.1	SQL gating + cubit locked
REQ-003	1.1, 2.1	unit fórmula + SQL trigger
REQ-004	1.3	unit publicación y archivo

Proposal: add-facturacion

Deriva del caso "Sistema de Facturación". Ejemplo Complejo: máquina de estados con transiciones reguladas.

Impacto (Impact Report)

- Features afectadas: ninguna; feature nueva `invoicing`
- Reutilizable: patrón CRUD; formato de moneda en core
- Supabase: tablas `invoices`, `invoice_items`, `payments`, `credit_notes`; contador secuencial atómico
- DI / rutas: +2 cubits (editor, listado), +3 páginas
- Riesgos: números duplicados por concurrencia (RT001); estados inválidos por doble tap

Why (Problema)

El negocio cobra sin trazabilidad: facturas manuales, pagos sin registro y vencimientos que nadie controla.

What Changes (Solución)

Emisión de facturas con máquina de estados (borrador→emitida→enviada→pagada/vencida/anulada), pagos totales/parciales, notas de crédito por excedente y numeración secuencial por emisor.

Capabilities

New Capabilities

- `invoicing`: ciclo de vida completo de facturas y pagos

Scope (Alcance)

Incluye: crear/editar borrador, emitir, enviar, registrar pagos parciales, vencimiento automático, anular con nota de crédito. **No incluye:** facturas recurrentes (RN012 → MODIFIED futuro), plantillas, reportes financieros, validación fiscal externa. **Dependencias:** catálogo de clientes, autenticación. **Suposiciones:** IVA 21% nacional / 0% exportación (RN013). **Preguntas abiertas:** ¿moneda múltiple? → una sola moneda v1.

Actores y permisos

Actor	Puede	No puede	Mapeo RLS
Emisor	CRUD SUS facturas/pagos	Ver facturas ajenas; editar número (RS002)	<code>auth.uid() = issuer_id</code>
Cliente	Ver estado de SUS facturas	Editar nada	<code>client_id = auth.uid()</code>
Cron	Marcar vencidas	—	security definer

Impact

- Código: ~16 ficheros en `lib/features/invoicing/`
- Datos: 4 tablas + RPC emitir/registrar_pago/marcar_vencidas
- Breaking changes: ninguno

Spec: invoicing

WHY

Cobrar exige documentos trazables con estados claros y pagos conciliados.

Purpose

Garantizar que cada transición de estado cumpla sus reglas y que la numeración sea secuencial e inmutable.

ADDED Requirements

Requirement: Crear y emitir factura (REQ-001)

Scenario: Emisión válida

- **WHEN** el emisor emite un borrador con ≥ 1 ítem y vencimiento posterior a emisión
- **THEN** recibe número secuencial FFF-000001 (RN001, RT001) y pasa a `emitida`

Scenario: Sin ítems

- **IF** el borrador no tiene ítems
- **THEN** el sistema DEBERÁ mostrar "No se puede emitir una factura sin ítems" (RN002)

Scenario: Vencimiento inválido

- **IF** $\text{fechaVencimiento} \leq \text{fechaEmision}$
- **THEN** el sistema DEBERÁ mostrar "La fecha de vencimiento debe ser posterior" (RN003)

Requirement: Transiciones de estado (REQ-002)

Scenario: Editar no-borrador

- **IF** la factura no está en `borrador`
- **THEN** el sistema DEBERÁ rechazar la edición (RN004)

Scenario: Pagar solo lo pagable

- **MIENTRAS** el estado sea `emitida` o `enviada`
- **EL SISTEMA DEBERÁ** aceptar pagos; cualquier otro estado los rechaza (RN006)

Requirement: Pagos y saldo (REQ-003)

Scenario: Pago parcial

- **WHEN** se registra un pago menor al total
- **THEN** la factura permanece con saldo pendiente visible

Scenario: Pago total

- **WHEN** $\Sigma \text{pagos} \geq \text{total}$
- **THEN** el estado pasa a `pagada` (RN008)...

Scenario: Sobre pago

- **IF** $\Sigma \text{pagos} > \text{total}$
- **THEN** se genera nota de crédito por el excedente (RN009) referenciando factura emitida (RN010)

Requirement: Vencimiento automático (REQ-004)

Scenario: Marcar vencida

- **CUANDO** $\text{fechaVencimiento} < \text{hoy}$ y $\text{estado} \in \{\text{emitida}, \text{enviada}\}$
- **EL SISTEMA DEBERÁ** transicionar a `vencida` (cron diario, RN007)

Requirement: Anulación (REQ-005)

Scenario: Anular factura

- **WHEN** el emisor anula una factura emitida/enviada/vencida
- **THEN** queda **anulada** y el monto como crédito a favor del cliente (RN011)

Requirement: Aislamiento (REQ-006)

- **ENTONCES** cada emisor SOLO ve sus facturas (RS001); el número es columna protegida no editable por frontend (RS002)

Design: add-facturacion

Context

Máquina de estados regulada. La fuente de verdad de las transiciones es la BD; el dominio las replica para validación temprana y tests.

Goals / Non-Goals

- Goals: cero estados inválidos; numeración sin huecos ni duplicados
- Non-Goals: recurrentes, plantillas, reportes

Decisions

D1: Transiciones en RPC transaccional

- Decisión: `rpc.emitir_factura`, `rpc.registrar_pago`, `rpc.anular` validan estado+reglas atómicamente; constraint CHECK de enum en BD
- Alternativa descartada: transiciones solo en Cubit (doble tap = doble pago)
- Por qué: concurrencia y auditoría

D2: Número secuencial con contador por emisor

- Decisión: tabla `invoice_sequences(issuer_id, next_number)` actualizada dentro del mismo RPC; columna `number` generada server-side
- Por qué: RN001 + RT001 + RS002 (no editable desde cliente)

D3: Estados como sealed class en domain

- Decisión: `InvoiceStatus` enum + guard `canTransitionTo()` puro
- Por qué: UI deshabilita acciones inválidas sin roundtrip

Ficheros afectados (resumen)

Elemento	Capa	Archivo
Invoice, InvoiceItem, Payment, CreditNote (+InvoiceStatus)	domain/entities	lib/features/invoicing/domain/entities/
InvoiceRepository	domain/repositories	...
CreateInvoice, UpdateDraft, IssueInvoice, SendInvoice, RegisterPayment, CancelInvoice, ListInvoices	domain/usecases	...
Models, RemoteDataSource, Impl	data	...
InvoiceEditorCubit, InvoiceListCubit (+states)	presentation/cubit	...
InvoiceEditorPage, InvoiceListPage, InvoiceDetailPage	presentation/pages	...
Migración 0010 (tablas + sequences + RPCs + RLS + cron vencidas)	supabase/migrations	...

Contratos Dart clave

```
abstract interface class InvoiceRepository {  
  Future<Either<Failure, Invoice>> createDraft(CreateInvoiceParams params);  
  Future<Either<Failure, Invoice>> updateDraft(String id, UpdateInvoiceParams params);  
  Future<Either<Failure, Invoice>> issue(String invoiceId);  
  Future<Either<Failure, Invoice>> send(String invoiceId);  
  Future<Either<Failure, PaymentResult>> registerPayment(RegisterPaymentParams params);  
  Future<Either<Failure, Invoice>> cancel(String invoiceId, String reason);  
  Future<Either<Failure, List<Invoice>>> listByIssuer({InvoiceStatus? status});  
}
```

Flujo de datos (registrar pago)

```
InvoiceDetailPage → InvoiceEditorCubit.pay(amount)  
  ▼  
RegisterPayment → rpc.registrar_pago(factura_id, monto)  
  | valida RN006 → insert payment → recalcula saldo  
  | └ saldo > 0 → Either.right(PartiallyPaid(saldo))  
  | └ saldo ≤ 0 → pasa 'pagada'; si sobra crea credit_note (RN009/RN010)  
  ▼  
UI refresca estado y lista de pagos
```

Backend Supabase

- invoices(+status enum check), invoice_items(check cantidad>0), payments, credit_notes, invoice_sequences
- RPC transaccionales ×3 + función cron `marcar_vencidas()`
- RLS issuer/cliente según tabla; number protegido (RS002)

Boundaries

No implementar recurrentes ni plantillas en este cambio; no exponer número editable.

Tasks: add-facturacion

1. Dominio

- ☐ 1.1 Entities + InvoiceStatus con canTransitionTo() (máquina de estados pura) Éxito: todas las transiciones válidas/inválidas cubiertas por unit test
- ☐ 1.2 Interface InvoiceRepository
- ☐ 1.3 UseCases ×7

2. Capa de datos + Backend Supabase

- ☐ 2.1 Migración 0010: tablas, sequences, RPCs ×3, RLS, cron vencidas
- ☐ 2.2 Models roundtrip (money como num; enums)
- ☐ 2.3 RemoteDataSource (RPC-first; sin UPDATE directo de estado desde cliente)
- ☐ 2.4 RepositoryImpl (mapeo errores SQL → Failure con mensajes RN00x)

3. Estado y presentación

- ☐ 3.1 EditorCubit/ListCubit + states sealed Éxito: doble tap en "pagar" no duplica pago (debounce + RPC idempotente)
- ☐ 3.2 Pages: editor (borrador), listado con filtros por estado, detalle con pagos

4. Integración

- ☐ 4.1 DI + rutas

5. Tests

- ☐ 5.1 Unit: máquina de estados completa + cálculo saldo/sobrepago
- ☐ 5.2 SQL: numeración concurrente (dos emisiones paralelas), pagos parciales→total
- ☐ 5.3 Widget: editor valida antes de emitir; detalle muestra saldo

Trazabilidad

Req	Tarea(s)	Test
REQ-001	1.1, 2.1	unit+SQL numeración
REQ-002	1.1, 1.3	unit transiciones
REQ-003	2.1, 1.3	SQL pagos parciales/sobrepago
REQ-004	2.1 (cron)	integration
REQ-005	2.1, 1.3	unit anulación
REQ-006	2.1	SQL RLS

Proposal: add-delivery-seguimiento

Deriva del caso "App de Delivery". Ejemplo Complejo: multi-actor, realtime (GPS) y geolocalización.

Impacto (Impact Report)

- Features afectadas: ninguna; feature nueva `delivery` (se asume catálogo restaurantes/menú existente)
- Reutilizable: auth por roles, patrón Either
- Supabase: tabla `orders` con máquina de estados amplia; Realtime Broadcast para GPS
- DI / rutas: +3 cubits (cliente, restaurante, repartidor), +4 páginas
- Riesgos: batería por frecuencia GPS; fugas de ubicación entre pedidos

Why (Problema)

Los pedidos se coordinan por teléfono y el cliente no sabe dónde está su comida.

What Changes (Solución)

Ciclo de pedido con estados (pendiente→...→entregado), aceptación/rechazo del restaurante con timeout 3 min, asignación de repartidor cercano, tarifa PostGIS y tracking GPS en tiempo real.

Capabilities

New Capabilities

- `order-lifecycle`: creación y transiciones de estado del pedido
- `delivery-tracking`: asignación y seguimiento GPS en vivo

Scope (Alcance)

Incluye: hacer/cancelar pedido, flujo restaurante (aceptar/preparar/listo), flujo repartidor (aceptar/recoger/entregar), tarifa \$1.5/km, tracking cada 3s. **No incluye:** pasarela de pagos real (mock), calificaciones (RN008/009 → MODIFIED futuro), re-asignación automática completa (solo alerta RN010). **Dependencias:** autenticación multi-rol, mapa (google_maps ya en pubspec). **Suposiciones:** radio de entrega 5 km; repartidor emite GPS solo durante pedido activo. **Preguntas abiertas:** ¿offline del repartidor? → v1 requiere conexión.

Actores y permisos

Actor	Puede	No puede	Mapeo RLS
Cliente	Pedir, cancelar en estados iniciales, rastrear SU pedido	Ver otros pedidos	<code>auth.uid() = customer_id</code>
Restaurante	Gestionar SU menú y pedidos entrantes	Cambiar estados post-recogido	<code>restaurant_id = auth.uid()</code>
Repartidor	Aceptar entregas, actualizar SU ubicación	Tener 2 pedidos activos (RN001)	locations: <code>auth.uid() = courier_id</code>

Impact

- Código: ~20 ficheros en `lib/features/delivery/`
- Datos: orders, order_items, courier_locations + RPC tarifa/asignación; channel realtime
- Breaking changes: ninguno

Spec: order-lifecycle + delivery-tracking

WHY

El pedido necesita estados claros entre tres actores y el cliente necesita ver el avance en vivo.

Purpose

Transiciones válidas por actor con timeouts regulados y streaming de ubicación privado.

ADDED Requirements

Requirement: Hacer y cancelar pedido (REQ-001)

Scenario: Pedido válido

- **WHEN** el cliente hace checkout con items disponibles dentro del radio de 5 km (RN011/RN012)
- **THEN** el pedido queda **pendiente** con tarifa = 1.5 €/km calculada server-side (RN005, RT002)

Scenario: Item no disponible

- **IF** algún item no está disponible
- **THEN** el sistema DEBERÁ mostrar "Producto no disponible" sin crear el pedido

Scenario: Cancelación tardía

- **IF** el estado ya no es **pendiente** ni **confirmado**
- **THEN** la cancelación DEBERÁ rechazarse (RN002)

Requirement: Timeout del restaurante (REQ-002)

Scenario: Aceptación a tiempo

- **WHEN** el restaurante acepta dentro de los 3 minutos
- **THEN** el pedido pasa a **confirmado**

Scenario: Sin respuesta

- **IF** pasan 3 min en **pendiente**
- **THEN** el cron DEBERÁ cancelarlo automáticamente (RN004)

Requirement: Asignación de repartidor (REQ-003)

Scenario: Repartidor acepta

- **WHEN** un repartidor a <2 km acepta (RN006) sin pedido activo (RN001)
- **THEN** queda asignado y el pedido pasa al flujo de entrega

Scenario: Doble pedido

- **IF** el repartidor tiene un pedido activo
- **THEN** el sistema DEBERÁ ocultar nuevos pedidos disponibles

Requirement: Tracking GPS (REQ-004)

Scenario: Emisión periódica

- **MIENTRAS** haya pedido activo asignado
- **EL REPARTIDOR DEBERÁ** publicar su ubicación cada 3s vía Realtime Broadcast (RT001)

Scenario: Privacidad de ubicación

- **ENTONCES** solo el cliente de ESE pedido ve esa ubicación (RS001); el repartidor solo escribe la suya (RS002)

Scenario: Entrega

- **WHEN** se marca **entregado**

- **THEN** cesa la emisión GPS y se cierra el canal

Design: add-delivery-seguimiento

Context

Realtime-first. El GPS usa Broadcast (efímero, sin persistir cada punto); la asignación usa RPC transaccional.

Goals / Non-Goals

- Goals: tracking fluido y privado; transiciones imposibles de corromper
- Non-Goals: pagos reales, calificaciones, re-asignación automática

Decisions

D1: Broadcast para GPS, tabla solo para "última ubicación"

- Decisión: canal `order:{id}` con Broadcast; `courier_locations` guarda el último punto para reconexiones
- Por qué: RT001 — Broadcast evita write-amplification en BD

D2: Tarifa vía RPC PostGIS

- Decisión: `rpc.calcular_tarifa(origin, destination)` con `ST_DWithin/ST_Distance`
- Por qué: RT002 + RN005/RN012 en servidor (no manipulable)

D3: Timeout con pg_cron + campo deadline

- Decisión: `accept_deadline` al crear; cron cada minuto cancela vencidos
- Por qué: RN003/RN004 sin dependencias externas

Ficheros afectados (resumen)

Elemento	Capa	Archivo
Order, OrderItem, Location (VO), CourierLocation	domain/entities	lib/features/delivery/domain/entities/
OrderRepository, DeliveryRepository (con Streams)	domain/repositories	...
PlaceOrder, CancelOrder, AcceptOrder, MarkX ×4, WatchOrder, WatchRestaurantOrders, AcceptDelivery, UpdateLocation, CalculateFee	domain/usecases	...
Models, Remote+RealtimeDataSource, Impls	data	...
CustomerOrderCubit, RestaurantOrdersCubit, TrackingCubit (+states)	presentation/cubit	...
CheckoutPage, RestaurantOrdersPage, AvailableDeliveriesPage, TrackingMapPage	presentation/pages	...
Migración 0011 (orders + locations + RPCs + RLS + cron)	supabase/migrations	...

Contratos Dart clave

```
abstract interface class OrderRepository {
  Future<Either<Failure, Order>> placeOrder(PlaceOrderParams params);
  Future<Either<Failure, Order>> transition({required String orderId, required OrderEvent event});
  Stream<Either<Failure, Order>> watchOrder(String orderId);
}
abstract interface class DeliveryRepository {
  Future<Either<Failure, double>> calculateDeliveryFee(Location a, Location b);
  Future<Either<Failure, Unit>> updateLocation(String orderId, Location loc);
  Stream<Location> watchCourierLocation(String orderId);
}
sealed class TrackingState {}
class TrackingLive extends TrackingState { final Location courier; const TrackingLive(this.courier); }
```

Flujo de datos (tracking)

```
TrackingCubit.start(orderId)
  └─> watchOrder (Realtime: cambios de estado del pedido)
    └─> watchCourierLocation (Broadcast channel order:{id})
Repartidor: geolocator stream (3s) → UpdateLocation → broadcast + upsert último punto
Cliente: mapa pinta TrackingLive(courier) → entregado → cancelar suscripciones
```

Backend Supabase

- orders(+status enum amplio, accept_deadline), order_items(check disponible), courier_locations
- RLS: cliente lee SU pedido; restaurante los SUyos; courier_locations write owner-only, read solo clientes con pedido activo de ese courier (policy por join)
- Realtime: habilitar broadcast; cron timeout

Boundaries

No integrar pasarela real ni calificaciones aquí; GPS nunca persiste histórico completo (privacidad).

Tasks: add-delivery-seguimiento

1. Dominio

- ☐ 1.1 Entities + OrderStatus (9 estados) con transiciones por actor
- ☐ 1.2 Interfaces OrderRepository/DeliveryRepository (incl. Streams)
- ☐ 1.3 UseCases ×10

2. Capa de datos + Backend Supabase

- ☐ 2.1 Migración 0011: tablas, RPCs tarifa/asignación, RLS, cron timeout
- ☐ 2.2 Models roundtrip (Location VO ↔ PostGIS lat/Ing)
- ☐ 2.3 RealtimeDataSource (watch order + broadcast GPS) y RemoteDataSource
- ☐ 2.4 RepositoryImpls

3. Estado y presentación

- ☐ 3.1 CustomerOrderCubit, RestaurantOrdersCubit, TrackingCubit Éxito: TrackingLive actualiza <3s; cancelación de streams al entregar
- ☐ 3.2 CheckoutPage, RestaurantOrdersPage, AvailableDeliveriesPage, TrackingMapPage Restricción: geolocator con permisos y battery-aware (pausa en background)

4. Integración

- ☐ 4.1 DI + rutas por rol

5. Tests

- ☐ 5.1 Unit: máquina de estados por actor; RN001/RN002/RN005/RN006/RN011/RN012
- ☐ 5.2 SQL: asignación concurrente (2 couriers), timeout cron, tarifa PostGIS
- ☐ 5.3 Widget: mapa con ubicación mock; estados del pedido

Trazabilidad

Req	Tarea(s)	Test
REQ-001	1.1, 1.3, 2.1	unit+SQL
REQ-002	2.1 (cron)	integration
REQ-003	2.1, 1.3	SQL asignación concurrente
REQ-004	2.3, 3.1, 3.2	cubit+widget streams

Proposal: add-buyers

Deriva del ejemplo "Gestión de compradores (Buyers)" del módulo histórico (contexto: app de rifas). Ejemplo **Simple**→**Intermedia**: CRUD con una regla de negocio.

Impacto (Impact Report)

- Features afectadas: ninguna; feature nueva `buyers` sobre módulo `raffles`
- Reutilizable: patrón listado+search existente; auth organizador
- Supabase: tablas `buyers`, `tickets` (ya existe `raffles`)
- DI / rutas: +1 cubit, +1 página, +1 ruta `/raffles/:id/buyers`
- Riesgos: bajo — aprobación de tickets debe ser idempotente

Why (Problema)

El organizador no puede gestionar quién compró tickets ni aprobar participantes seleccionados; todo vive en chats y notas sueltas.

What Changes (Solución)

Listado de compradores por rifa con búsqueda por nombre/teléfono, aprobación de tickets seleccionados y liberación de los no seleccionados.

Capabilities

New Capabilities

- `buyers-management` : gestión de compradores y estados de ticket

Scope (Alcance)

Incluye: listar compradores, buscar por nombre o teléfono, aprobar tickets, liberar no seleccionados. **No incluye:** notificaciones, pagos, edición de datos del comprador. **Dependencias:** autenticación (identidad del organizador), feature raffles. **Suposiciones:** un comprador pertenece a una sola rifa. **Preguntas abiertas:** ¿reaprobar un aprobado? → idempotente, sin efecto; ¿aprobación atómica? → SÍ, RPC transaccional.

Actores y permisos

Actor	Puede	No puede	Mapeo RLS
Organizador	Ver/aprobar/liberar tickets de SUS rifas	Ver rifas ajenas	<code>raffle.owner_id = auth.uid()</code>
Comprador	— (fuera de alcance v1)	—	—

Impact

- Código: ~9 ficheros en `lib/features/buyers/`
- Datos: buyers + FK a raffles; RPC aprobar/lote liberar
- Breaking changes: ninguno

Spec: buyers-management

WHY

El organizador necesita una vista única de compradores para decidir qué tickets quedan confirmados.

Purpose

Listado filtrable con acciones de aprobación/liberación seguras e idempotentes.

ADDED Requirements

Requirement: Listar compradores (REQ-001)

Scenario: Listado por rifa

- **WHEN** el organizador abre los compradores de su rifa
- **THEN** ve nombre, teléfono, cantidad de tickets y estado

Scenario: Rifa ajena

- **IF** la rifa no pertenece al organizador autenticado
- **THEN** Supabase DEBERÁ devolver vacío vía RLS

Requirement: Buscar compradores (REQ-002)

Scenario: Búsqueda por prefijo

- **CUANDO** el organizador escribe ≥ 3 caracteres
- **EL SISTEMA DEBERÁ** filtrar por nombre o teléfono (búsqueda delegada al backend)

Requirement: Aprobar tickets (REQ-003)

Scenario: Aprobación exitosa

- **WHEN** el organizador aprueba los tickets seleccionados de un comprador
- **THEN** pasan a `aprobado` en una transacción atómica (RPC)

Scenario: Re-aprobación idempotente

- **IF** el ticket ya estaba aprobado
- **THEN** la operación DEBERÁ no tener efecto ni error

Requirement: Liberar tickets (REQ-004)

Scenario: Liberación masiva

- **WHEN** el organizador libera los tickets NO seleccionados
- **THEN** solo esos pasan a `libre` y quedan disponibles para venta

Design: add-buyers

Context

Cambio pequeño sobre el módulo raffles existente. Complejidad Intermedia: design ligero, una puerta combinada.

Goals / Non-Goals

- Goals: aprobación atómica e idempotente; búsqueda sin traer todo al cliente
- Non-Goals: notificaciones, pagos, edición de compradores

Decisions

D1: Aprobación y liberación vía RPC transaccional

- Decisión: `rpc.aprobar_tickets(buyer_id, ticket_ids[])` y `rpc.liberar_no_seleccionados(raffle_id)`
- Por qué: atomicidad (spec lo exige) y RLS respetada en servidor

D2: Búsqueda con ilike en RPC

- Decisión: filtrado server-side (`nombre ilike %q% or telefono ilike %q%`)
- Por qué: consistencia con RT del módulo histórico — nunca filtrar listas grandes en cliente

Ficheros afectados

Elemento	Capa	Archivo
Buyer, TicketStatus	domain/entities	lib/features/buyers/domain/entities/buyer.dart
BuyerRepository	domain/repositories	.../domain/repositories/buyer_repository.dart
GetBuyers, ApproveTickets, ReleaseUnselected	domain/usecases	.../domain/usecases/
BuyerModel	data/model	.../data/models/buyer_model.dart
BuyerRemoteDataSource	data/datasource	...
BuyerRepositoryImpl	data/repositories	...
BuyersCubit + BuyersState	presentation/cubit	...
BuyersPage (+search bar)	presentation/pages	.../presentation/pages/buyers_page.dart
DI + ruta anidada bajo raffles	core	service_locator.dart · app_router.dart
Migración 0012 (buyers + RPCs)	supabase/migrations	...

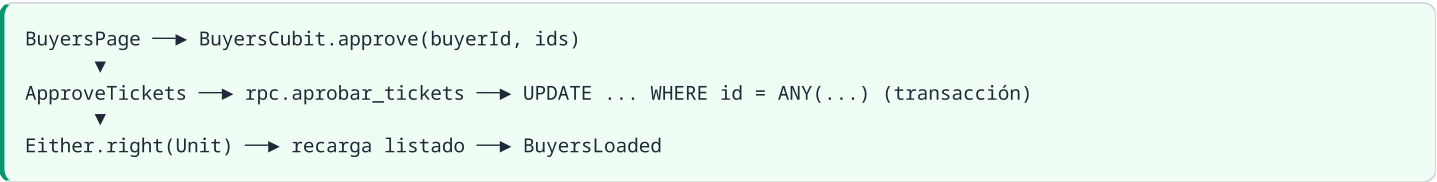
Contratos Dart clave

```
abstract interface class BuyerRepository {
  Future<Either<Failure, List<Buyer>>> getBuyers({required String raffleId, String? query});
  Future<Either<Failure, Unit>> approveTickets({required String buyerId, required List<String> ticketIds});
  Future<Either<Failure, Unit>> releaseUnselected({required String raffleId});
}

sealed class BuyersState {}

class BuyersLoaded extends BuyersState { final List<Buyer> buyers; const BuyersLoaded(this.buyers); }
```

Flujo de datos (aprobar)



Backend Supabase

- buyers(id, raffle_id→raffles, nombre, telefono)
- tickets ya existente: +estado enum(libre,vendido,aprobado); FK buyer opcional
- RPCs ×2 security definer; RLS organizador por join a raffles

Boundaries

No tocar la feature raffles salvo la columna de estado indicada.

Tasks: add-buyers

Complejidad Intermedia: una sola puerta combinada antes de implementar (proposal+spec+design+tasks revisados juntos).

1. Dominio

- ☐ 1.1 Entity Buyer + TicketStatus enum
- ☐ 1.2 Interface BuyerRepository
- ☐ 1.3 UseCases ×3

2. Capa de datos + Backend Supabase

- ☐ 2.1 Migración 0012: tabla buyers + RPCs ×2 + RLS por organizador
- ☐ 2.2 BuyerModel roundtrip
- ☐ 2.3 RemoteDataSource (RPC + query con búsqueda server-side)
- ☐ 2.4 BuyerRepositoryImpl (mapeo excepciones → Failure)

3. Estado y presentación

- ☐ 3.1 BuyersState sealed + BuyersCubit
- ☐ 3.2 BuyersPage con search bar y acciones aprobar/liberar

4. Integración

- ☐ 4.1 DI + ruta anidada /raffles/:id/buyers

5. Tests

- ☐ 5.1 Unit usecases (query vacía, ids vacíos → failure)
- ☐ 5.2 SQL: idempotencia de aprobación; RLS rifa ajena
- ☐ 5.3 Widget: listado, búsqueda, snackbars de resultado

Trazabilidad

Req	Tarea(s)	Test
REQ-001	2.1, 3.2	SQL RLS
REQ-002	2.1, 2.3	unit datasource
REQ-003	2.1, 1.3	SQL idempotencia
REQ-004	2.1, 1.3	SQL liberación